



Network Change Validation with Relational NetKAT

HAN XU, Princeton University, United States

ZACHARY KINCAID, Princeton University, United States

RATUL MAHAJAN, University of Washington, United States

DAVID WALKER, Princeton University, United States

Relational NetKAT (RN) is a new specification language for network change validation. Engineers use RN to specify intended changes by providing a trace relation R , which maps existing packet traces in the pre-change network to intended packet traces in the post-change network. The *intended* set of traces may then be checked against the *actual* post-change traces to uncover errors in implementation. Trace relations are constructed compositionally from a language of combinators that include trace insertion, trace deletion, and packet transformation, as well as regular operators for concatenation, union, and iteration of relations. We provide algorithms for converting trace relations into a new form of NetKAT transducer and also for constructing an automaton that recognizes the image of a NetKAT automaton under a NetKAT transducer. These algorithms, together with existing decision procedures for NetKAT automaton equivalence, suffice for validating network changes. We provide a denotational semantics for our specification language, prove our compilation algorithms correct, implement a tool for network change validation, and evaluate it on a set of benchmarks drawn from a production network and Amazon's Batfish toolkit.

CCS Concepts: • **Theory of computation** → **Formal languages and automata theory**; • **Networks**;

Additional Key Words and Phrases: Network verification; Change validation; NetKAT; Automata theory

ACM Reference Format:

Han Xu, Zachary Kincaid, Ratul Mahajan, and David Walker. 2026. Network Change Validation with Relational NetKAT. *Proc. ACM Program. Lang.* 10, POPL, Article 14 (January 2026), 29 pages. <https://doi.org/10.1145/3776656>

1 Introduction

As industrial networks have grown in size and complexity, so too has the difficulty of keeping them running smoothly, and when networks go down, critical services of all kinds are disrupted. Recognizing the threat, researchers looked to formal methods to systematically detect and prevent problems, and over the course of the 2010s, the foundations for sound and scalable network verification were born through systems such as Anteater [Mai et al. 2011], Header Space Analysis [Kazemian et al. 2012], Veriflow [Khurshid et al. 2013], Batfish [Fogel et al. 2015], NetKAT [Anderson et al. 2014a], Minesweeper [Beckett et al. 2017], and Self-Starter [Kakarla et al. 2020], among others. Today, thanks to the transfer of technology, ideas, and people, most if not all large hyperscale cloud providers, including Microsoft, Amazon, and Google, use verification tools to make their networks more reliable [Backes et al. 2019; Jayaraman et al. 2019; Zeng et al. 2014].

Despite the successes and wide adoption of these techniques, making changes to network configurations continues to be a risky activity. A key problem is that *existing specification languages are poorly suited for concisely describing the full effect of common network changes*. They focus on

Authors' Contact Information: Han Xu, hx3501@princeton.edu, Princeton University, United States; Zachary Kincaid, zkincaid@cs.princeton.edu, Princeton University, United States; Ratul Mahajan, ratul@cs.washington.edu, University of Washington, United States; David Walker, dpw@princeton.edu, Princeton University, United States.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 2475-1421/2026/1-ART14

<https://doi.org/10.1145/3776656>

single-snapshot verification, that is, checking the properties of a *single* network configuration. Using single-snapshot verification, one can check coarse properties of the changed network such as reachability, isolation, or access control. However, a comprehensive analysis of a change will often involve (1) identifying all of the paths in the old network that satisfy some property (the need to be changed), (2) checking that the new network contains all and exactly the corresponding changed paths and that those changed paths have the desired properties, and (3) checking that none of the other paths in the old network (those that should not have been altered) have been corrupted, blocked, or rerouted in the new network. Single-snapshot verifiers provide no means of “getting one’s hands on” the old paths and ensuring they are properly changed in the updated network—by definition single-snapshot verifiers do not reference the old paths or old network at all.

To overcome such issues, Xu et al. [2024] recently proposed *relational network verification*. In their system, called Relat, networks were modeled as sets of *paths*, where each path is a sequence of locations (device groups, switches, or interfaces) along which a packet will travel. A language of *rational relations* over paths was used in combination with the old network to specify the intended network change. More precisely, network engineers would write a transformer R to indicate the intent of a change and specifications took (roughly) the form “old $\triangleright R =$ new” where old and new encoded the paths through the old and new networks respectively. The relation R transforms the old network into a set of expectations and those expectations are compared with the new network. Their change specifications were compact, often being just 10s of lines of code, even for networks with millions of flows, because even in large networks, individual changes are often small.

While Relat is a strong progress on an important problem, its “set of paths” model is a weak model for a modern network. It cannot represent internal decision-making made by network forwarding tables, which classify and act on packets with different headers in different ways, nor can it represent packet transformations, such as those used in network address translators (NATs), tunnels, or other purposes in software-defined networks (SDN) (e.g., in-band network telemetry).

In this paper, we introduce Relational NetKAT (RN for short), a new language capable of fully and accurately specifying changes to stateless modern networks. Whereas Relat uses regular sets of paths to describe networks, we adopt a stronger base model—that of NetKAT [Anderson et al. 2014a]—in which networks are described as sets of *traces*. Here, a *trace* is a sequence of *located packets*—that is the headers of packets (destination IP, source IP, etc.) together with their network location. By using NetKAT to describe traces, one can easily model packet forwarding tables, with their match-action semantics compactly, as well as the various ways that stateless modern networks modify packets including in devices such as stateless NATs, SDN devices, and tunnels.

Relational NetKAT augments traditional NetKAT with a new sublanguage of trace relations R . Users can write relations that specify desired network changes by inserting new subpaths into existing traces, or deleting old ones that should no longer be present. Alternatively, users can map packet relations across the elements of a trace to represent changes to packet contents between old and new networks. This new language of trace relations is compositional, conferring the ability to construct complex specifications from simpler parts through the use of the regular operators for concatenation, union, and iteration, and has a well-defined denotational semantics.

The main technical contribution of this paper is the automata-theoretic machinery that enables verification of relational NetKAT specifications. First, we introduce a class of *NetKAT transducers*, which are used to represent trace relations. NetKAT transducers may be thought of as an extension of NetKAT automata with an additional tape. Second, we give an algorithm for constructing a NetKAT automaton that recognizes the image of a NetKAT automaton under a NetKAT transducer. In principle, this can be accomplished using classical algorithms for transducers, but the size of the resulting NetKAT automaton is exponential in the size of the input. We avoid this state-space explosion by making careful use of our domain: While networks are large, most changes are

intended to be small [Xu et al. 2024], affecting a limited number of locations and/or flows in the network. Consequently, “most” of the trace relation between old and new networks is the identity relation, and we take care to exploit this property during image computation.

We have proven the correctness of our compilation algorithms and implemented them in a tool for network verification. We evaluate the effectiveness of our tool in multiple ways. First, we demonstrate the use of Relational NetKAT on Rela’s open benchmark, which is drawn from an example change implemented in one of Alibaba’s production networks. While our prototype tool is considerably slower than Rela due to the much more expressive trace specifications we process,¹ we demonstrate it is fast enough for large-scale industrial use. Moreover, the fact one may process all packet equivalence classes in parallel means that the slowdown may have little practical significance. Second, we demonstrate the expressiveness of the RN specification language by tackling a series of examples drawn from the Batfish toolkit. Here, we craft relational specifications for a corporate network with NATs and IP tunneling, demonstrating how changes to forwarding may be specified and checked.

2 Background: (Modestly Extended) NetKAT

NetKAT [Anderson et al. 2014b] is a decidable language for specifying properties of stateless network data planes. While there are a number of frameworks for verifying stateless data planes, some in heavy use in industry, NetKAT stands out as it provides a general, compositional specification language in which new properties are easily constructed from smaller, simpler parts. When first presented, NetKAT was equipped with an equational theory proven sound and complete with respect to its denotational semantics. In this paper, we ignore NetKAT’s equational theory and focus entirely on its denotational semantics and algorithms for compilation to automata. We need only these latter elements for clearly defined specifications and decidable verification procedures.

In this paper, we present a slight variant and minor refactoring of traditional NetKAT, which we call “Modestly Extended” NetKAT when we need to point out the differences. Modestly Extended NetKAT contains a few additional primitives for crafting relations between one packet and the next. These extensions are not very important for NetKAT itself; they play a larger role in our new Relational NetKAT. Readers familiar with NetKAT may skip ahead to the syntax and semantics presented in Figure 1.

Networks as Sets of Traces. NetKAT models a network as a set of possible *traces*, where each trace captures the sequence of steps a packet takes from some network source to some network sink. More specifically, a *trace* (tr) is a sequence of *located packets* (pk), where a *located packet* is record mapping packet header field names to values and mapping the special *loc* field to the packet’s current location in the network. Networks locations are often identified using a switch and a port on that switch [Anderson et al. 2014b]. For simplicity, most of our examples are presented at a slightly higher level of abstraction, calling out only the switch at which a packet has just arrived, not the port at which it sits. For instance, if pk_1 is $\{\text{loc} = A; \text{dst.ip} = 1.1.1.1; \dots\}$ and pk_2 and pk_3 are similar but with location fields B and C respectively then the trace $(pk_1 \ pk_2 \ pk_3)$ represents a path starting at the ingress to A , through B , and ending at the ingress to C for a packet with destination IP address 1.1.1.1. In this case, the packet contents remain unchanged as it travels that path, but in general, packet contents may change as it travels from A to B . We write $pk[f \leftarrow v]$ to update field f of pk with value v .

Two traces may be *concatenated* only when the last packet of one trace equals the first packet of the next trace, and when they do, the equal packets are dropped from the result. For instance,

¹and less time optimizing our prototype—Rela uses a heavily optimized, off-the-shelf library for equivalence of ordinary finite automata.

concatenating $(pk_1 \ pk_2 \ pk_3)$ with $(pk_4 \ pk_5 \ pk_6)$ results in $(pk_1 \ pk_2 \ pk_5 \ pk_6)$ when $pk_3 = pk_4$ and is undefined otherwise. Hence, concatenating two 2-element traces leaves us with a 2-element trace. The concatenation of two trace sets $(S_1 \circ S_2)$ is defined as follows

$$S_1 \circ S_2 = \{pk_{11} \ pk_{12} \ \cdots \ pk_{1(n-1)} \ pk_{22} \ \cdots \ pk_{2m} \mid \\ pk_{11} \ \cdots \ pk_{1(n-1)} \ pk_{1n} \in S_1, pk_{21} \ pk_{22} \ \cdots \ pk_{2m} \in S_2, pk_{1n} = pk_{21}\}$$

Compact Network Specifications. For a network of even modest size, its set of traces is enormous as each packet header may be 100+ bits and paths can vary in length. Fortunately, NetKAT provides a means to specify these sets of paths compactly. NetKAT contains two essential sublanguages: (1) a language of boolean predicates, and (2) a language of packet relations.

The boolean predicates (*pred*) are used to classify packets so we can describe how sets of related packets are forwarded efficiently. The basic predicate $(f = v)$ is true of packets with field f containing value v . Boolean combinations of basic predicates are possible using conjunction ($\cdot$), disjunction ($+$) and negation ($\neg$). For instance, $(src.ip = 1.1.1.1) \cdot (loc = A + loc = C)$ describes the set of packets with source IP 1.1.1.1 at locations A or C . We write $(f \neq v)$ for $\neg(f = v)$.

The packet relations (*PkR*) explain how to forward a packet one hop along its journey through a network. Semantically, such relations denote sets of pairs of located packets. Traditional NetKAT contains a basic relation to update a field of a packet, write $f \leftarrow v$. When f is the location field, $loc \leftarrow L$ models the movement of a packet from its current location to some (likely new) location L . NetKAT also contains the identity relation ($\bar{I}$), which relates every packet to itself, and the empty relation ($\bar{0}$). More complex relations can be constructed through composition of packet relations ($PkR_1 \circ PkR_2$), intersection of packet relations ($PkR_1 \cap PkR_2$), and cartesian product of predicates ($pred_1 \times pred_2$). The latter relates every packet in the set $pred_1$ with every packet in the set $pred_2$ —it may be used to model a nondeterministic forwarding relation, for instance. The cartesian product is a new primitive for NetKAT, though its semantics could already be encoded through a disjunction of many alternatives. We choose to add it as a primitive because doing so leads to a more efficient implementation for a variety of our specifications. Without it, our encodings would be quadratic in the size of the sets represented by $pred_1$ and $pred_2$. Of particular importance is the *havoc* relation, which we define as 1×1 , and which relates any two packets. We write $\overline{pred}$ for the subset of the identity relation that includes only packets satisfying *pred*. In this version of NetKAT, $\overline{pred}$ is an abbreviation for $\bar{I} \cap (pred \times pred)$.

Full NetKAT expressions (K) denote sets of traces, with each trace at least two elements long. NetKAT expressions include packet relations (*PkR*), which give rise to sets of 2-element traces, concatenation ($K_1 \circ K_2$), union ($K_1 + K_2$), and iteration of trace sets (K^*). Finally, *dup* is the set of 3-element traces: $\{pk \ pk \ pk \mid pk \in Pk\}$. While the concatenation of any trace *tr* of length n with a 2-element trace leaves a trace with n elements, concatenation of an n -element trace with *dup* leaves a trace with $n + 1$ elements. One common abbreviation used in our specifications is *alltraces(pred)*—the set of all traces that satisfy *pred* on every hop. *alltraces(pred)* is defined as $(pred \times 1 \circ dup)^* \circ pred \times pred$. The syntax and semantics of NetKAT is summarized in Figure 1.

Examples. The data plane of a network is composed of a set of *forwarding tables*—for simplicity here, one forwarding table per switch. Each forwarding table consists of a series of *match-action rules*. The "match" component of a given rule is modeled as a NetKAT predicate and the action is modeled by a packet relation. For example, a forwarding table containing rules

- (1) forward packets with `dst.ip 1.0.0.0` to switch B
- (2) modify packets with `dst.ip 1.0.0.1` so their `src.ip` is 2.0.0.0 before forwarding to C
- (3) drop all other packets

Syntax:

$$\begin{aligned}
Pred & ::= 0 \mid 1 \mid f = v \mid Pred_1 + Pred_2 \mid Pred_1 \cdot Pred_2 \mid \neg Pred \\
Pkr & ::= \bar{0} \mid \bar{1} \mid f \leftarrow v \mid Pred \times Pred \mid Pkr_1 \circ Pkr_2 \mid Pkr_1 \cup Pkr_2 \mid Pkr_1 \cap Pkr_2 \mid \neg Pkr \\
K & ::= Pkr \mid K_1 + K_2 \mid K_1 \circ K_2 \mid K^* \mid dup
\end{aligned}$$

Common Abbreviations:

$$\begin{aligned}
havoc & = 1 \times 1 \\
\overline{pred} & = \bar{1} \cap (pred \times pred) \\
alltraces(pred) & = (pred \times 1 \circ dup)^* \circ pred \times pred \\
alltraces & = alltraces(1)
\end{aligned}$$

Semantics:

$$\begin{aligned}
\llbracket 0 \rrbracket_{Pred} & = \emptyset \\
\llbracket 1 \rrbracket_{Pred} & = \{pk \mid pk \in Pk\} \\
\llbracket loc = v \rrbracket_{Pred} & = \{pk \mid pk.loc = v\} \\
\llbracket \neg Pred \rrbracket_{Pred} & = Pk \setminus \llbracket Pred \rrbracket_{Pred} \\
\llbracket Pred_1 + Pred_2 \rrbracket_{Pred} & = \llbracket Pred_1 \rrbracket_{Pred} \cup \llbracket Pred_2 \rrbracket_{Pred} \\
\llbracket Pred_1 \cdot Pred_2 \rrbracket_{Pred} & = \llbracket Pred_1 \rrbracket_{Pred} \cap \llbracket Pred_2 \rrbracket_{Pred} \\
\llbracket \bar{0} \rrbracket_{Pkr} & = \emptyset \\
\llbracket \bar{1} \rrbracket_{Pkr} & = \{(pk, pk) \mid pk \in Pk\} \\
\llbracket f \leftarrow v \rrbracket_{Pkr} & = \{(pk, pk[f \leftarrow v]) \mid pk \in Pk\} \\
\llbracket Pred_1 \times Pred_2 \rrbracket_{Pkr} & = \{(pk_1, pk_2) \mid pk_1 \in \llbracket Pred_1 \rrbracket_{Pred}, pk_2 \in \llbracket Pred_2 \rrbracket_{Pred}\} \\
\llbracket Pkr_1 \circ Pkr_2 \rrbracket_{Pkr} & = \{(pk_1, pk_3) \mid \exists pk_2. (pk_1, pk_2) \in \llbracket Pkr_1 \rrbracket_{Pkr} \wedge (pk_2, pk_3) \in \llbracket Pkr_2 \rrbracket_{Pkr}\} \\
\llbracket \neg Pkr \rrbracket_{Pkr} & = Pk \times Pk \setminus \llbracket Pkr \rrbracket_{Pkr} \\
\llbracket Pkr_1 \cup Pkr_2 \rrbracket_{Pkr} & = \llbracket Pkr_1 \rrbracket_{Pkr} \cup \llbracket Pkr_2 \rrbracket_{Pkr} \\
\llbracket Pkr_1 \cap Pkr_2 \rrbracket_{Pkr} & = \llbracket Pkr_1 \rrbracket_{Pkr} \cap \llbracket Pkr_2 \rrbracket_{Pkr} \\
\llbracket Pkr \rrbracket_K & = \{pk_1 \, pk_2 \mid (pk_1, pk_2) \in \llbracket Pkr \rrbracket_{Pkr}\} \\
\llbracket K_1 + K_2 \rrbracket_K & = \llbracket K_1 \rrbracket_{Pkr} \cup \llbracket K_2 \rrbracket_{Pkr} \\
\llbracket K_1 \circ K_2 \rrbracket_K & = \llbracket K_1 \rrbracket_{Pkr} \cdot \llbracket K_2 \rrbracket_{Pkr} \\
\llbracket K^* \rrbracket_K & = \bigcup_{n \geq 0} \llbracket K^n \rrbracket_K, \quad \text{where } K^0 = \bar{1}, K^{n+1} = K^n \circ K \\
\llbracket dup \rrbracket_K & = \{pk \, pk \, pk \mid pk \in Pk\}
\end{aligned}$$

Fig. 1. Syntax and Semantics of (Modestly Extended) NetKAT

may be modeled using the NetKAT expression $FTA =$

$$\begin{aligned}
& \overline{(dst.ip = 1.0.0.0 \circ loc \leftarrow B)} + & \% \text{ rule 1} \\
& \overline{(dst.ip = 1.0.0.1 \circ src.ip \leftarrow 2.0.0.0 \circ loc \leftarrow C)} + & \% \text{ rule 2} \\
& \overline{(dst.ip \neq 1.0.0.0 \cdot dst.ip \neq 1.0.0.1 \circ \bar{0})} & \% \text{ rule 3}
\end{aligned}$$

FTA's denotation contains a set of 2-element traces, including for instance, the trace $\{loc = L; dst.ip = 1.0.0.0; \dots\} \{loc = B; dst.ip = 1.0.0.0; \dots\}$ for all locations L since the position of the

forwarding table in the network is not specified. Further, since $\llbracket K \circ \bar{0} \rrbracket_K = \llbracket \bar{0} \rrbracket_K$, the third rule is unnecessary.

Now, suppose forwarding table FTA is located at switch A and switches B and C have their own forwarding tables FTB and FTC respectively. A network containing those three switches may be modeled with the following expression K_{net} .

$$(((\overline{\text{loc} = A} \circ \text{FTA}) + (\overline{\text{loc} = B} \circ \text{FTB}) + (\overline{\text{loc} = C} \circ \text{FTC})) \circ \text{dup})^*$$

Intuitively, the expression states that a packet travelling through the network may encounter any of switches A, B or C, and when it does it is processed using their forwarding table. The Kleene star indicates a packet may encounter 0 or more switches along its journey through the network. The *dup* expression is used to control what is observed as a packet travels through the network—it makes a copy of a packet and adds that copy to the trace. Without it, we would have only 2-element traces that record start and end points of a packet's journey. When making a change to a network, engineers often care about the details of the path being used, for instance, in cases where the change involves decommissioning an internal switch, eliminating internal congestion, or obeying a security that mandates only certain devices (perhaps in certain countries) are used.

Single-snapshot Verification with NetKAT. NetKAT has regularly been used as a *single-snapshot* specification and verification platform. To do so, one uses the techniques of the previous paragraphs to encode the network of interest as a NetKAT expression K_{net} . Next, to specify a property, one adds constraints to K_{net} and constructs an equation or inequation. For example, to check that a particular class of packets, say those satisfying predicate *blocked*, are not forwarded through the network, one might check the equation $\llbracket \overline{\text{blocked}} \circ K_{net} \rrbracket_K = \llbracket \bar{0} \rrbracket_K$. To check that some packets satisfying *reach* will be forwarded from A to C, one might check $\llbracket \overline{\text{loc} = A} \circ \text{reach} \circ K_{net} \circ \overline{\text{loc} = C} \rrbracket_K \neq \llbracket \bar{0} \rrbracket_K$.

Change Validation with NetKAT. To our knowledge, NetKAT and its variants have only ever been used for single-snapshot verification. However, we observe that due to the nature of its equational specifications, NetKAT can validate the simplest kinds of changes—it can compare subsets of the old, pre-change network to the new post-change network with an equation of the form $K_{pre} \cap \text{subset} = K_{post} \cap \text{subset}$, using the intersection operator of KATch [Moeller et al. 2024]. More general change validation is out of reach for NetKAT. There is no general way to identify a set of traces in the pre-change network, apply a transformation to them, and then check that all and exactly the transformed traces appear as desired in the post-change network. As we demonstrate via examples below, this capability is key to validating many types of changes.

3 Relational NetKAT

Relational NetKAT (RN) extends NetKAT with a new sublanguage for expressing *trace relations* R . Such trace relations denote subsets of $\text{Trace} \times \text{Trace}$ and may be used to capture fine-grained correspondences between traces that occur in a network before a change and traces that occur in a network after a change. These trace relations may be included in NetKAT expressions using the relational image operation $K \triangleright R$, which applies relation R to K yielding some new set of traces K' . Trace relations enable a new, modular style of network specification—one focused on transforming networks and comparing the transformations to desired specifications. This style is particularly well-suited to verification of changes to networks.

Normally, to use RN to verify a change, a network operator will supply a pair of trace relations—one to transform the pre-change network (R_{pre}) and one to transform the post-change network (R_{post}). Given those two specifications, the RN system:

- translates the pre-change network into a NetKAT expression K_{pre} ,

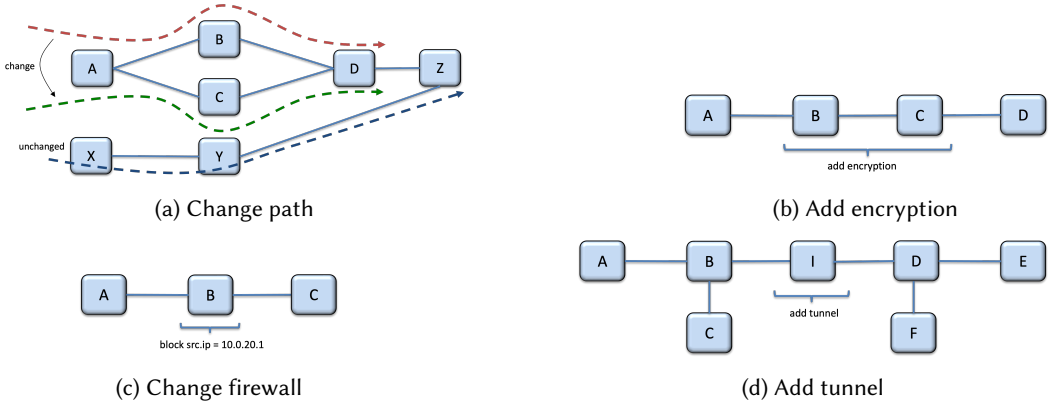


Fig. 2. Four network updates.

- translates the post-change network into a NetKAT expression K_{post} , and
- implements a decision procedure for the specification $K_{pre} \triangleright R_{pre} = K_{post} \triangleright R_{post}$, where $K \triangleright R$ is the set of traces computed by finding the image of R applied to K .

When the second relation, R_{post} is the identity, we are left with the equation $K_{pre} \triangleright R_{pre} = K_{post}$. In this case, R_{pre} represents the engineer's (possibly incorrect) beliefs about the change process, and $K_{pre} \triangleright R_{pre}$ specifies the expected impact of the change on the original network. On the other hand, K_{post} is the *actual* effect of the change. The goal is to uncover bugs by checking the engineer's beliefs against reality.

3.1 Trace Relations

To introduce and illustrate the use of trace relations in network change specifications, we provide a series of examples before supplying a formal semantics.

Example: Specifying a Path Change. Consider a simple scenario, presented in Figure 2a, in which a network engineer must take a switch (say, switch B) offline for maintenance to upgrade or replace it. Prior to doing so, any existing traffic traversing B should instead be redirected along another path, perhaps the one using switch C . A strong specification of such a change should include two elements:

- (1) Traces in the pre-change network that do not include B should stay the same in the post-change network.
- (2) Traces in the pre-change network that include B should instead use C in the post-change network, but should otherwise stay the same.

This specification is easy to state and understand precisely because it refers to the traces *from the old network*. If one attempted a single-snapshot specification of the property (i.e., a specification that does not include direct reference to “the pre-change network”), one would effectively have to enumerate all of the (possibly millions) of paths in the pre-change network by hand, or by some other process, patching those paths by replacing B with C where necessary. Alternatively—and this has been the modus operandi of past NetKAT verification efforts, which have not focused on change validation—one checks weaker properties such as reachability. However, in industrial practice, the stronger properties are needed to have confidence in the change. Hence, despite the availability of in-house verification tools, much change checking has, somewhat surprisingly, been done manually and at great cost in terms of human time and effort[Xu et al. 2024].

In RN, specifying and checking such changes is straightforward. To begin, consider part (1) of the specification above, which demands that a subset of the network *stay the same*. Ironically,

“staying the same” is amongst the most common components of our change specifications. Such specifications are instances of the *identity relation* on traces, written $Id(K)$, which relates every trace $t \in \llbracket K \rrbracket_K$ to itself. Hence, in this case, the first part of the specification may be written using the following trace relation.

$$stay_the_same = Id(alltraces(loc \neq B))$$

To implement the second part of the specification, we must transform traces from using B to using C instead. To this end, Relational NetKAT supplies $DeleteSeg(K)$ to delete a segment matching K and $InsertSeg(K')$ to add a segment matching K' . More precisely, $DeleteSeg(K)$ relates all traces in $\llbracket K \circ dup \circ havoc \rrbracket_K$ to an “empty” trace². Recall that the *havoc* relation leaves the relationship between one packet and the next in a trace completely unconstrained. Hence, the “ $dup \circ havoc$ ” extension may be read as saying “and any other packet may come next.” In our experience, when writing specifications, one typically writes them in “chunks,” where one chunk does not constrain the next; The $DeleteSeg(K)$ (also $InsertSeg(K')$) combinator conveniently admits this style as the following examples show.

Now that we have the combinators we need, changing a segment that uses B to one that uses C is achieved via the relation $DeleteSeg(loc \leftarrow B) \cdot InsertSeg(loc \leftarrow C)$, which “concatenates” an insertion after a deletion, giving us a replacement of one by the other. To extend these trace relations from one-hop relations to full paths, one can concatenate the desired prefix and/or suffix. Usually, we assume no loop in a desired network, hence part (2) of the change may be expressed in its entirety as follows.

$$change_path = Id(alltraces) \cdot DeleteSeg(loc \leftarrow B) \cdot InsertSeg(loc \leftarrow C) \cdot Id(alltraces)$$

With both parts (1) and (2) in hand, the full relation is constructed using a union.

$$fullchange = stay_the_same + change_path$$

To check this change, we simply apply $fullchange$ to the pre-change network and compare the result with the post-change network—that is, we verify the equation $K_{pre} \triangleright fullchange = K_{post}$.

Example: Encrypting payloads. Protocols such as IPsec will encrypt packet payloads to ensure client privacy. One way to model encryption is to extend packets with an additional field encrypted, set to 1 when the packet payload is encrypted and set to 0 when the packet payload is unencrypted. Now, consider the scenario presented in Figure 2b, where a network operator adds encryption and decryption protocols at the network endpoints. One might want to verify that packets traversing internal network node B and C are encrypted after the change, but otherwise act similarly, following similar paths, and with headers undisturbed.

Reasoning about such changes may be accomplished via the *Map* combinator, which is a generalization of *Id*, used earlier. $Map(PkR, K)$ applies packet relation PkR at each hop in every trace in $\llbracket K \rrbracket_K$; $Id(K)$ is an abbreviation for $Map(\bar{1}, K)$. In our case, relating any packet to its encrypted counterpart is achieved by the packet relation $(encrypted \leftarrow 1)$. Since we wish packets be encrypted when traversing nodes B and C , we should restrict the relation so it only applies in those locations: $(loc = B + loc = C) \circ (encrypted \leftarrow 1)$. Elsewhere in the network, packets should be unchanged: $\neg(loc = B + loc = C) \circ \bar{1}$. The desired transformation is the union of the two packet

²The “empty” trace is represented here as any length-1 trace.

relations applied universally:

$$\begin{aligned}
 \text{inside} &= \text{loc} = B + \text{loc} = C \\
 \text{outside} &= \neg \text{inside} \\
 \text{encrypt_it} &= \overline{\text{inside}} \circ (\text{encrypted} \leftarrow 1) \\
 \text{cleartext} &= \overline{\text{outside}} \circ \bar{1} \\
 \text{spec} &= \text{Map}(\text{encrypt_it} + \text{cleartext}, \text{alltraces})
 \end{aligned}$$

Example: Firewall. Firewalls are widely used in all kinds of networks. Common changes involve adding, deleting, or modifying firewall policies. Figure 2c presents a network where a firewall is placed at node B . The network administrator has decided to block all traffic with source IP address 10.0.20.1, due to uncertainty about whether such traffic is malicious. The goal is to verify that the firewall rule is correctly enforced at node B , while ensuring that all other traffic remains unaffected.

Such changes can be specified using the *Filter* combinator. Whereas *Map* applies a packet relation to every "hop" in a trace, *Filter* requires a particular packet relation hold at a specific point in a trace relation. In this case, for each trace tr_1, tr_2 from an element (tr_1, tr_2) of the *firewall_path* relation, defined below. Such traces must all include a packet at node B with a $\text{src.ip} \neq 10.0.20.1$. Other (tr_1, tr_2) pairs are excluded from *firewall_path*.

$$\text{firewall_path} = \text{Id}(\text{alltraces}) \cdot \text{Filter}(\overline{\text{loc} = B \cdot \text{src.ip} \neq 10.0.20.1}) \cdot \text{Id}(\text{alltraces})$$

To include all other traces that do not pass through B (thereby excluding just those traces that do pass through B but have $\text{src.ip} = 10.0.20.1$) we use the familiar identity relation:

$$\text{stay_the_same} = \text{Id}(\text{alltraces}(\text{loc} \neq B))$$

Combining *stay_the_same* and *firewall_path*, gives us the firewall update relation we desire: $K_{pre} \triangleright (\text{stay_the_same} + \text{firewall_path}) = K_{post}$, which asserts that the post-change network behaves like the pre-change network except for traffic passing through B , which is subject to the firewall rule.

Example: Tunneling. Figure 2d presents a system involving two corporate networks, one with nodes A, B, C , and the other with nodes D, E, F . The internet I sits between the two. We will assume that A, B, C use IP addresses identified by predicate net_1 and D, E, F use IP addresses identified by predicate net_2 . In this example, the user makes a change that adds a tunnel between C and D . The tunnel entry after C will add a new set of headers to every packet passing through it, with $\text{dst.ip} = 128.112.0.0$ (C 's destination IP). The tunnel exit prior to D will strip the header, revealing the original header underneath. A similar transformation occurs in the opposite direction (D and C) with IP 128.112.0.1 (D 's destination IP) used.

One of the limitations of NetKAT is that it demands all packets use the same set of headers throughout the network (header fields cannot be pushed and popped arbitrarily). Still, we can model a network that admits one extra layer of headers where needed by assuming all packets have a second copy of each header field. We name the two copies dst.ip_1 and dst.ip_2 with dst.ip_1 the inner header and dst.ip_2 the outer header. When dst.ip_2 is not in use, the field is set to 0. With this set-up, relating packets moving through the tunnel is achieved via *tunnel_change* =

$$\begin{aligned}
 (\text{loc} = I \cdot \text{net}_2 \cdot \text{dst.ip}_2 \leftarrow 128.112.0.0) + & \quad \% \text{ in the tunnel, headed to net2} \\
 (\text{loc} = I \cdot \text{net}_1 \cdot \text{dst.ip}_2 \leftarrow 128.112.0.1) & \quad \% \text{ in the tunnel, headed to net1}
 \end{aligned}$$

The relations for (1) all traces that travel through the tunnel one from one network to the other (*net2net_traces*), (2) all traces that remain in one network or the other, not using the tunnel

Syntax:

$$R ::= \text{Filter}(PkR) \mid \text{Map}(PkR, K) \mid \text{Delete}(K) \mid \text{Insert}(K) \mid R_1 \cdot R_2 \mid R_1 + R_2 \mid R^*$$

$$K ::= \dots \mid K \triangleright R$$

Abbreviations:

$$\begin{aligned} Id(K) &= \text{Map}(\bar{1}, K) \\ \text{DeleteSeg}(K) &= \text{Delete}(K \circ \text{dup} \circ \text{havoc}) \\ \text{InsertSeg}(K) &= \text{Insert}(K \circ \text{dup} \circ \text{havoc}) \\ \text{MapSeg}(PkR, K) &= \text{Map}(PkR, K) \cdot \text{Map}(\text{havoc}, \text{havoc}) \end{aligned}$$

Semantics:

$$\begin{aligned} \llbracket \text{Filter}(PkR) \rrbracket_R &= \{(pk_1, pk_2) \mid (pk_1, pk_2) \in \llbracket PkR \rrbracket_{PkR}\} \\ \llbracket \text{Map}(PkR, K) \rrbracket_R &= \{(pk_1 \dots pk_n, pk'_1 \dots pk'_n) \mid pk_1 \dots pk_n \in \llbracket K \rrbracket_K, \\ &\quad \forall i \in [1, n]. (pk_i, pk'_i) \in \llbracket PkR \rrbracket_{PkR}\} \\ \llbracket \text{Delete}(K) \rrbracket_R &= \{(pk_1 \dots pk_n, pk) \mid pk \in Pk, pk_1 \dots pk_n \in \llbracket K \rrbracket_K\} \\ \llbracket \text{Insert}(K) \rrbracket_R &= \{(pk, pk_1 \dots pk_n) \mid pk \in Pk, pk_1 \dots pk_n \in \llbracket K \rrbracket_K\} \\ \llbracket R_1 + R_2 \rrbracket_R &= \llbracket R_1 \rrbracket_R \cup \llbracket R_2 \rrbracket_R \\ \llbracket R_1 \cdot R_2 \rrbracket_R &= \left\{ (pk_1 \dots pk_{n_1} \dots pk_{n_2}, pk'_1 \dots pk'_{m_1} \dots pk'_{m_2}) \mid \right. \\ &\quad \left. (pk_1 \dots pk_{n_1}, pk'_1 \dots pk'_{m_1}) \in \llbracket R_1 \rrbracket_R, \right. \\ &\quad \left. (pk_{n_1} \dots pk_{n_2}, pk'_{m_1} \dots pk'_{m_2}) \in \llbracket R_2 \rrbracket_R \right\} \\ \llbracket R^* \rrbracket_R &= \bigcup_{n \geq 0} \llbracket R^n \rrbracket_R, \quad \text{where } R^0 = \text{Filter}(\text{havoc}), \quad R^{n+1} = R^n \cdot R \\ \llbracket K \triangleright R \rrbracket_K &= \{tr' \mid tr \in \llbracket K \rrbracket_K, (tr, tr') \in \llbracket R \rrbracket_R, |tr'| \geq 2\} \end{aligned}$$

Fig. 3. Syntax and Semantics of Trace Relations.

(*other_traces*), and (3) the overall specification (*spec*) are presented below.

$$\begin{aligned} \text{net2net_traces} &= Id(\text{alltraces}) \cdot \text{MapSeg}(\text{tunnel_change}, \text{loc} = I) \cdot Id(\text{alltraces}) \\ \text{other_traces} &= Id(\text{alltraces}(\text{loc} \neq I)) \\ \text{spec} &= \text{net2net_traces} + \text{other_traces} \end{aligned}$$

Summary. These examples, while idealized, illustrate a number of the (subjective) benefits of this style of specification:

- *Change specifications are clear:* Specifications say what changes directly as a relation between old and new, and say what does not change equally clearly.
- *Specifications are compact:* Specifications are usually proportional the size of a change rather than proportional to the number of devices, forwarding rules, or paths in a network. Specifications for real changes in networks in our study in Section 5 are only slightly longer than the idealized variants presented here, since most of the time, most of the network says unchanged.

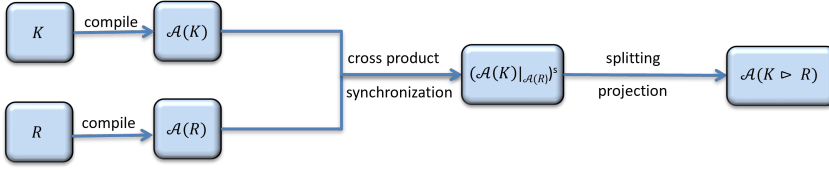


Fig. 4. Compilation process of Relational NetKAT

- *Specifications are expressive*: Using regular languages over trace relations and four simple but expressive atomic primitives, we can encode a wide variety of change-validation properties—many of which cannot be captured in prior work [Xu et al. 2024].
- *Specifications are compositional*: Sub-regions of a network may be specified separately then composed with specifications for other sub-regions.
- *Specifications are modular*: The specification need not be heavily “intermixed” with elements that represent parts of the post-change (or pre-change) networks. The specification can be defined separately from the network to which it is applied. A single specification could be reused and applied to multiple networks, or parts of the same network over time, if similar changes occur.

The formal syntax and semantics of trace relations, as well as some common abbreviations we use, are summarized in Figure 3. While most of the semantics is unsurprising, there are a couple of subtle technicalities to note. In particular, an element of a relation (tr_1, tr_2) may contain a length-1 trace. For example, a filter is implemented as a pair of length-1 traces. When a relation R is applied to a set of traces K , yielding a new set of traces, any remaining length-1 traces are excluded from the result. Concatenation in the sublanguage is also subtly different: It eliminates one intermediate packet rather than two, as in NetKAT.

4 Automata

The Relational NetKAT compiler architecture is presented in Figure 4 and proceeds as follows.

- (1) The NetKAT expression K is compiled into a standard NetKAT automaton, denoted $\mathcal{A}(K)$.
- (2) The Relational expression R is compiled into a NetKAT transducer, denoted $\mathcal{A}(R)$.
- (3) These two automata are combined to form a transducer $(\mathcal{A}(R)|_{\mathcal{A}(K)})^S$, which captures the synchronized behavior of K and R .
- (4) Finally, the transducer $(\mathcal{A}(R)|_{\mathcal{A}(K)})^S$ is transformed into a projected automaton $\mathcal{A}(K \triangleright R)$ that recognizes $\llbracket K \triangleright R \rrbracket_K$ by applying projection. While doing so, a key *relational splitting* optimization decomposes relational transitions to reduce the state space.

We establish the correctness of this compilation process by showing that the behavior of the final automaton $\mathcal{A}(K \triangleright R)$ aligns with the denotational semantics of $K \triangleright R$.

4.1 Automaton Model for NetKAT Expression K and Relation R

Our automata model for NetKAT expressions K are similar to prior work [Foster et al. 2015; Smolka et al. 2015], following the Antimirov derivatives construction as used in work [Smolka et al. 2015] to generate finite automata from NetKAT programs. Since our language equivalence decision procedure for K requires one step of determinization, we also adopt the explicit state representation from this line of work.

The key difference in our automata is that we unify the two different kinds of transitions: ϵ -transitions that lead to accepting states, and δ -transitions that lead to non-final states into one

unified transition Δ representing all of the cases. Using this Δ transition, we further give a different definition of the transition relation $(s_0, pk_0) \xrightarrow{w} (s_n, pk_n)$, characterizing the language acceptance.

This unified Δ transition-based formulation and Antimirov derivatives construction make the compilation process readily extensible to relational expressions R , enabling us to define a uniform cross-product construction between automata for K and R in the subsequent steps.

NetKAT Automata. We begin by defining a general automaton model that captures the structure needed to represent NetKAT programs.

DEFINITION 1. A **NetKAT automaton** is a tuple $M = (S, S_0, S_f, \Delta)$ where S is a finite set of states, $S_0 \subseteq S$ is the set of start states, $S_f \subseteq S$ is the set of final states, and $\Delta : S \times S \rightarrow 2^{Pk \times Pk}$ is a transition relation.

Unlike classical automata, where transitions depend solely on the current input symbol, network devices operate on the entire packet and update it in place. Upon receiving a packet, a device inspects and possibly modifies it before forwarding. To accurately model this behavior, each transition in a NetKAT automaton involves a configuration pair (s, pk) , consisting of a state and a packet.

To distinguish packets on different tapes in the relational setting, throughout this section we use x and w to denote packets and traces on the NetKAT automaton tape, and y and z for those on the other tape of the NetKAT transducer.

The transition relation of M is defined inductively over packet traces. A labeled transition

$$(s_0, x_0) \xrightarrow{w} (s_n, x_n)$$

indicates that, starting from state s_0 and input packet x_0 , the automaton can process the trace w and reach state s_n with final packet x_n .

- **Base case:** $(s_0, x_0) \xrightarrow{\epsilon} (s_0, x_0)$
- **Inductive case:** If $(x_0, x_1) \in \Delta(s_0, s_1)$ and $(s_1, x_1) \xrightarrow{w} (s_n, x_n)$, then $(s_0, x_0) \xrightarrow{x_1 w} (s_n, x_n)$.

The language accepted by M is the set of input/output traces that begin from an initial state and terminate in an accepting state:

$$L(M) = \{x_0 w \mid s_0 \in S_0, s_f \in S_f, x_0, x_n \in Pk. (s_0, x_0) \xrightarrow{w} (s_f, x_n)\}.$$

Compilation of K . To compile a NetKAT expression K into an automaton, we adopt the Antimirov derivative approach [Smolka et al. 2015], adapted to our explicit automaton model. Although this construction yields a non-deterministic (i.e., the out transition of each state may not be disjoint) automaton, it guarantees that the number of states is bounded by the syntactic length of the expression $l(K) + 1$ (with proof in Appendix), a desirable property inherited from the derivative framework [Antimirov 1996; Sakarovitch 2009]. The compilation process is straightforward and is detailed in the appendix. We also formally prove its correctness:

THEOREM 4.1. For all NetKAT expressions K , let $\mathcal{A}(K)$ be the automaton constructed via the Antimirov derivative. Then:

$$L(\mathcal{A}(K)) = \llbracket K \rrbracket_K.$$

NetKAT Transducer. NetKAT transducers model the semantics of relational NetKAT programs R , which operate on *pairs* of traces. Intuitively, a NetKAT transducer behaves like a two-tape automaton, where each tape corresponds to a trace in the pair. Each state in the automaton is implicitly associated with the current packet being processed on each tape.

In our syntax, we support four kinds of atomic relational constructs: $Map(PkR, K)$ advances both tapes simultaneously, $Delete(K)$ advances only the left tape, $Insert(K)$ advances only the right tape, and $Filter(PkR)$ without advancing either tape.

To model the behavior of these constructs, the NetKAT transducer includes four basic transition types, corresponding directly to the above operations:

DEFINITION 2. A **NetKAT transducer** is a tuple $T = (S, S_0, S_f, \Delta_S, \Delta_L, \Delta_R, \Delta_E)$ where S is a finite set of states, $S_0 \subseteq S$ is a set of start states, $S_f \subseteq S$ is a set of final states, $\Delta_S : S \times S \rightarrow (Pk \times Pk) \times (Pk \times Pk)$ is a synchronous transition relation, $\Delta_L : S \times S \rightarrow Pk \times Pk$ is an asynchronous left transition relation, $\Delta_R : S \times S \rightarrow Pk \times Pk$ is an asynchronous right transition relation, and $\Delta_E : S \times S \rightarrow Pk \times Pk$ is an epsilon transition relation.

Similar to NetKAT automaton M , the transition relation of T is defined inductively over pairs of packet traces. A labeled transition of the form:

$$(s_0, (x_0, y_0)) \xrightarrow{(w,z)} (s_n, (x_{n_1}, y_{n_2}))$$

indicates that, starting from (x_0, y_0) , the transducer produces the output traces w and z along a path to state s_n with final packets (x_{n_1}, y_{n_2}) .

- **Base case:** $(s_0, (x_0, y_0)) \xrightarrow{(\epsilon, \epsilon)} (s_0, (x_0, y_0))$.
- **Both tapes move:** If $((x_0, y_0), (x_1, y_1)) \in \Delta_S(s_0, s_1)$ and $(s_1, (x_1, y_1)) \xrightarrow{(w,z)} (s_n, (x_{n_1}, y_{n_2}))$, then $(s_0, (x_0, y_0)) \xrightarrow{(x_1 w, y_1 z)} (s_n, (x_{n_1}, y_{n_2}))$.
- **First tape only:** If $(x_0, x_1) \in \Delta_L(s_0, s_1)$ and $(s_1, (x_1, y_1)) \xrightarrow{(w,z)} (s_n, (x_{n_1}, y_{n_2}))$, then $(s_0, (x_0, y_0)) \xrightarrow{(x_1 w, z)} (s_n, (x_{n_1}, y_{n_2}))$.
- **Second tape only:** If $(y_0, y_1) \in \Delta_R(s_0, s_1)$ and $(s_1, (x_0, y_1)) \xrightarrow{(w,z)} (s_n, (x_{n_1}, y_{n_2}))$, then $(s_0, (x_0, y_0)) \xrightarrow{(w, y_1 z)} (s_n, (x_{n_1}, y_{n_2}))$.
- **No tape moves:** If $(x_0, y_0) \in \Delta_E(s_0, s_1)$ and $(s_1, (x_0, y_0)) \xrightarrow{(w,z)} (s_n, (x_{n_1}, y_{n_2}))$, then $(s_0, (x_0, y_0)) \xrightarrow{(w,z)} (s_n, (x_{n_1}, y_{n_2}))$.

The language accepted by the NetKAT transducer T is then defined as the set of trace pairs processed from a start state to an accepting state:

$$L(T) = \{(xw, yz) \mid s_0 \in S_0, s_f \in S_f, x, x', y, y' \in Pk, (s_0, (x, y)) \xrightarrow{(w,z)} (s_f, (x', y'))\}.$$

Correctness of Compilation. Similar to the case for NetKAT expressions, we can ensure the correctness of the NetKAT transducer with respect to the denotational semantics. The number of states is again bounded by the syntactic length of the expression $l(R) + 1$. The full details of compilation and proof is in the Appendix.

THEOREM 4.2. For all relational NetKAT expressions R , let $\mathcal{A}(R)$ be the transducer constructed via the Antimirov derivative. Then:

$$L(\mathcal{A}(R)) = \llbracket R \rrbracket_R.$$

4.2 Cross Product and Synchronization

Our goal in this section is to compile an automaton that faithfully captures the semantics of the projection expression $K \triangleright R$. As an intermediate step, we construct the cross-product transducer $\mathcal{A}(R)|_{\mathcal{A}(K)}$, which combines the behavior of the NetKAT automaton $\mathcal{A}(K)$ and the NetKAT transducer $\mathcal{A}(R)$. We then eliminate all forms of asynchronous and ϵ transitions from $\mathcal{A}(R)|_{\mathcal{A}(K)}$ to form a synchronized version of the NetKAT transducer $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$. We break the compilation process into two steps:

- (1) **Cross Product:** Construct the cross-product transducer $\mathcal{A}(R)|_{\mathcal{A}(K)}$ from $\mathcal{A}(K)$ and $\mathcal{A}(R)$ by aligning transitions from $\mathcal{A}(K)$ with the first tape of $\mathcal{A}(R)$. The resulting transducer $\mathcal{A}(R)|_{\mathcal{A}(K)}$ accepts the relation $\{(w, z) \in \mathcal{A}(R) \mid w \in \mathcal{A}(K)\}$.
- (2) **Synchronization:** Eliminate all Δ_E , Δ_L , and Δ_R transitions to obtain the synchronized transducer $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$, in which transitions occur simultaneously on both tapes. Note that $L((\mathcal{A}(R)|_{\mathcal{A}(K)})^s) \neq L(\mathcal{A}(R)|_{\mathcal{A}(K)})$; however, they agree on the second tape of valid NetKAT traces (length greater than 2):

$$\{z \mid (w, z) \in (\mathcal{A}(R)|_{\mathcal{A}(K)})^s, |z| \geq 2\} = \{z \mid (w, z) \in \mathcal{A}(R)|_{\mathcal{A}(K)}, |z| \geq 2\}.$$

Cross Product Construction. Suppose we have NetKAT automaton $\mathcal{A}(K) = (S_k, S_{k0}, S_{kf}, \Delta)$ that accepts single traces, while the NetKAT transducer $\mathcal{A}(R) = (S_r, S_{r0}, S_{rf}, \Delta_S, \Delta_L, \Delta_R, \Delta_E)$ accepts pairs of traces. The purpose of the cross-product $\mathcal{A}(R)|_{\mathcal{A}(K)}$ is to match every trace on the first tape of $\mathcal{A}(R)$ to a valid trace accepted by $\mathcal{A}(K)$.

We define the cross-product transducer $\mathcal{A}(R)|_{\mathcal{A}(K)} = (S_{kr}, S_{kr0}, S_{krf}, \Delta'_S, \Delta'_L, \Delta'_R, \Delta'_E)$ as follows:

- $S_{kr} = S_k \times S_r$ is the set of composite states.
- $S_{kr0} = S_{k0} \times S_{r0}$ is the set of initial state pairs.
- $S_{krf} = S_{kf} \times S_{rf}$ is the set of accepting state pairs.

The transition relation is defined as follows:

- **Synchronized transitions on both tapes:**

$$\Delta'_S((s_k, s_r), (s'_k, s'_r)) = \{((x_1, y_1), (x_2, y_2)) \mid (x_1, x_2) \in \Delta(s_k, s'_k) \wedge ((x_1, x_2), (y_1, y_2)) \in \Delta_S(s_r, s'_r)\}$$

- **Transition on the first tape only:**

$$\Delta'_L((s_k, s_r), (s'_k, s'_r)) = \{(x_1, x_2) \mid (x_1, x_2) \in \Delta(s_k, s'_k) \wedge (x_1, x_2) \in \Delta_L(s_r, s'_r)\}$$

- **Transition on the second tape only:**

$$\Delta'_R((s_k, s_r), (s'_k, s'_r)) = \begin{cases} \Delta_R(s_r, s'_r) & \text{if } s_k = s'_k \\ \emptyset & \text{otherwise} \end{cases}$$

- **Transition on no tape:**

$$\Delta'_E((s_k, s_r), (s'_k, s'_r)) = \begin{cases} \Delta_E(s_r, s'_r) & \text{if } s_k = s'_k \\ \emptyset & \text{otherwise} \end{cases}$$

That is, when the relational transition consumes a packet on the X tape, we synchronize it with a corresponding transition in $\mathcal{A}(K)$. When the relational transition doesn't consume on the X tape, we keep the NetKAT state unchanged (i.e., $s_k = s'_k$) and perform only the transition on NetKAT transducer states.

Synchronization. After constructing the intermediate transducer $\mathcal{A}(R)|_{\mathcal{A}(K)}$, we eliminate all transitions involving Δ_E , Δ_L , and Δ_R , yielding a standard two-tape NetKAT transducer $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$ in which each transition consumes exactly one packet from both tapes. Formally, we refine $\mathcal{A}(R)|_{\mathcal{A}(K)}$ from:

$$(S_{kr}, S_{kr0}, S_{krf}, \Delta'_S, \Delta'_L, \Delta'_R, \Delta'_E) \quad \text{to} \quad (S_{kr}, S_{kr0}, S_{krf}, \Delta_S^*, \emptyset, \emptyset, \emptyset).$$

where Δ_S^* represent the synchronized transition for the new transducer.

A key observation is that synchronization does not preserve the full trace language:

$$L((\mathcal{A}(R)|_{\mathcal{A}(K)})^s) \neq L(\mathcal{A}(R)|_{\mathcal{A}(K)}).$$

However, since the projection $K \triangleright R$ only concerns the trace sequence on the *second tape*, we need only preserve the second-tape behavior. That is, we require:

$$\{z \mid (w, z) \in L((\mathcal{A}(R)|_{\mathcal{A}(K)})^s), |z| \geq 2\} = \{z \mid (w, z) \in L(\mathcal{A}(R)|_{\mathcal{A}(K)}), |z| \geq 2\}.$$

To achieve this, we apply a two-step strategy:

- **Simulate right-only transitions:** For every transition in Δ'_R that advances the second tape while leaving the first unchanged, we replace it with a synchronized transition:

$$\Delta_{SR}(s_1, s_2) = \{((x_1, y_1), (x_1, y_2)) \mid (y_1, y_2) \in \Delta'_R(s_1, s_2)\}.$$

This construction preserves the effect of the original right-only transition while achieving synchronization by introducing a no-op (i.e., static) move on the first tape.

- **ϵ -Elimination:** We compute the transitive closure of Δ'_E (transitions that move neither tape) and Δ'_L (transitions that move only the first tape). Using this closure, we propagate the effects of these transitions into Δ'_S and Δ_{SR} , effectively incorporating all intermediate ϵ -moves into synchronized transitions. This process mirrors standard ϵ -elimination in automata theory and yields a fully synchronized transducer.

The resulting NetKAT transducer $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$ is compatible with the projection operator and preserves the desired second-tape behavior. Full algorithmic details of the synchronization and ϵ -elimination procedures are deferred to the appendix. We summarize the correctness of the construction with the following theorem:

THEOREM 4.3. *Let $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$ be the synchronized NetKAT transducer constructed from the cross-product and synchronization procedures. Then:*

$$\{z \mid (w, z) \in L((\mathcal{A}(R)|_{\mathcal{A}(K)})^s)\} = \llbracket K \triangleright R \rrbracket_K.$$

Once $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$ is constructed, we apply standard reachability analysis to prune unreachable states and transitions. This optimization is important for performance in practice, as shown in our evaluation in Section 5, though we defer the technical details to the appendix.

4.3 Relational Splitting and Projection

The ultimate goal of this section is to project the NetKAT transducer $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$ into the final form $\mathcal{A}(K \triangleright R)$, while avoiding state explosion.

High-Level Idea. Following the synchronization procedure, we obtain a NetKAT transducer $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$ where every transition consumes a pair of packets—one from each tape. Our objective now is to project out the first tape X and extract the trace set from the second tape Y .

A naive approach to projection is to explicitly track the first-tape packet as part of the state. That is, we construct a NetKAT automaton where the states are of the form (s, pk) and where $\Delta((s, x), (s', x')) \triangleq \{(y, y') : ((x, y), (x', y')) \in \Delta_S(s, s')\}$. Thus the configuration space of the NetKAT transducer $S \times (Pk \times Pk)$ is isomorphic to the configuration space of the NetKAT automaton $(S \times Pk) \times Pk$, but in the NetKAT automaton the first packet in the configuration is part of the explicit state rather than the symbolic state.

However, this strategy leads to an exponential blowup in the state space. For each automaton state s , we would need to generate up to $|Pk|$ copies to account for all possible packet values pk , resulting in an infeasible number of states. For instance, even a packet with just 32-bit source and destination IP fields would yield at least 2^{64} distinct combinations—making equivalence-checking impossible in practice.

A key observation is that, in practice, relational NetKAT programs typically induce only "small" changes while preserving most of the structure at each transition. For example, suppose a NetKAT

relation R is of the form $Id(K)$ for some NetKAT expression K . In this case, we need not track the entire packet state on the X tape, because it is always equal to the packet on the Y tape!

Thus, for this example, we can construct a NetKAT automaton that recognizes the projection of the relation $\llbracket Id(K) \rrbracket_R$ using a NetKAT transducer T with the same state space as $\mathcal{A}(Id(K))$, and with the transition relation

$$\Delta(s, s') \triangleq \{(y, y') \mid ((x, y), (x', y')) \in \Delta_S(s, s'), x = y, x' = y'\}.$$

What makes this optimization possible is that, in all transitions $(s_1, (x_1, y_1)) \xrightarrow{(x_2, y_2)} (s_2, (x_2, y_2))$ of $\mathcal{A}(R)$, the value of x is completely determined by s and y . While this is not true for all NetKAT transducers, we can always partition the transitions of $\mathcal{A}(R)$ into finitely many cases in which the Y -tape value determines the X -tape value. Although the worst-case size of such an injective partition (i.e., partitioning until y determines x) is $O(|Pk|)$, our “small change” observation suggests that the size of this partition is often small—in the case of $Id(K)$, it has only one cell. The definition below formalizes our intuition about *efficiently projectable transducers*.

DEFINITION 3 (EFFICIENTLY PROJECTABLE TRANSDUCER). *Let $T = (S, S_0, S_f, \Delta_S, \emptyset, \emptyset, \emptyset)$ be a synchronous NetKAT transducer. Its projection $|T| \triangleq (S, S_0, S_f, \Delta)$ onto the second tape is a NetKAT automaton with the same state space, initial states, and final states as T , and where the transition relation is defined as:*

$$\Delta(s, s') \triangleq \{(y, y') \mid \exists x, x' \in Pk. ((x, y), (x', y')) \in \Delta_S(s, s')\}.$$

We say that T is *efficiently projectable* if:

$$L(|T|) = \{z \mid (w, z) \in L(T)\}.$$

Notice that $L(|T|) \supseteq \{z \mid (w, z) \in L(T)\}$ for any T , and that equality of the two sets holds only in special cases. One case in which a NetKAT transducer is efficiently projectable is when the y -packet uniquely determines its corresponding x -packet. Under this assumption, each configuration $(s, (x, y))$ of the original transducer is in one-to-one correspondence with (s, y) in the projected automaton. From a bisimulation perspective, this yields a strong invariant:

$$\exists x_1, x_2 \in Pk. (s_1, (x_1, y_1)) \xrightarrow{(x_2, y_2)}_T (s_2, (x_2, y_2)) \iff (s_1, y_1) \xrightarrow{y_2}_{|T|} (s_2, y_2).$$

However, in real-world settings, this uniqueness assumption is often too restrictive. Some fields—such as VLAN tags, source ports, or other metadata—may be abstracted or ignored, meaning y cannot fully determine x . Enforcing strict uniqueness in such cases would require explicitly enumerating all values of these abstracted fields, leading to an exponential blowup of size $2^{|f|}$, where $|f|$ is the length of hidden fields.

To avoid this explosion, we relax the uniqueness condition and introduce the notion of *x-bisimulation*. The observation here is that for hidden fields, all of the different values associated with them have identical behavior on the y -tape across all subsequent transitions. They are safe to collapse even if multiple x values may map to the same y . We relax the uniqueness condition to y uniquely determines a set of *behaviourally equivalent* values of x (with respect to the y -tape). Since the state s and the y -tape remain observable after projection, we only need to consider bisimulation of the x -tape for a fixed state s and y . This motivates the following definition.

DEFINITION 4 (X-BISIMULATION). *For a synchronous NetKAT transducer $T = (S, S_0, S_f, \Delta_S, \emptyset, \emptyset, \emptyset)$, a relation $x_1 \equiv_{s, y} x_2$ is an x -bisimulation if:*

- For all $x_1 \equiv_{s, y} x_2$ and $((x_1, y), (x_3, y')) \in \Delta_S(s, s')$, there exists x_4 such that $((x_2, y), (x_4, y')) \in \Delta_S(s, s')$ and $x_3 \equiv_{s', y'} x_4$.

- For all $x_1 \equiv_{s,y} x_2$ and $((x_2, y), (x_4, y')) \in \Delta_S(s, s')$, there exists x_3 such that $((x_1, y), (x_3, y')) \in \Delta_S(s, s')$ and $x_3 \equiv_{s',y'} x_4$.

We define the maximum x -bisimulation relation $R_{\max}$ to be the union of all x -bisimulations.

If the y -packet determines a set of x -packets that are behaviorally indistinguishable on the y -tape, then transitions in the original transducer involving (x, y) pairs can be faithfully simulated using only y . That is, for all reachable configurations (x_1, y_1) at state s_1 , the original transducer makes a transition

$$\exists x_1, x_2 \in \text{Pk.}(s_1, (x_1, y_1)) \xrightarrow{(x_2, y_2)} (s_2, (x_2, y_2))$$

if and only if the projected automaton makes a transition

$$(s_1, y_1) \xrightarrow{y_2} (s_2, y_2).$$

This effectively means that the set of reachable configurations $(s, (x, y))$ is bisimilar to (s, y) —and therefore, projection preserves the trace semantics.

We introduce the notion of *bisimulation witnesses* as a sufficient condition for bisimilarity of a synchronous NetKAT transducer T and its projection $|T|$. A bisimulation witness for the transducer T is a function $B : S \rightarrow 2^{Pk \times Pk}$ imply that there is a bisimulation relation $\sim$ between T and $|T|$ defined by

$$(s, (x, y)) \sim (s', y') \iff s = s', y = y', \text{ and } (x, y) \in B(s)$$

DEFINITION 5 (BISIMULATION WITNESS). Let $T = (S, S_0, S_f, \Delta_S, \emptyset, \emptyset, \emptyset)$ be a synchronous NetKAT transducer. A function $B : S \rightarrow 2^{Pk \times Pk}$ is a bisimulation witness if the following conditions holds:

- (1) *(Bisimulation Witness)* The relation $(x_1 \equiv_{s,y} x_2) \triangleq ((x_1, y) \in B(s)) \wedge ((x_2, y) \in B(s))$ is a x -bisimulation for transducer T .
- (2) *(Domain coverage)* For all $s \in S$, $B(s) \supseteq \bigcup_{s' \in S} \text{dom}(\Delta(s, s'))$
- (3) *(Range coverage)* For all $s \in S$, $B(s') \supseteq \bigcup_{s \in S} \text{range}(\Delta(s, s'))$

This bisimulation witness B captures the observable behavior of the transducer at each state. Condition (1) weakens the strict uniqueness requirement on x and replaces it with behavioral equivalence: all x values compatible with the same y must exhibit indistinguishable behavior on y -tape across transitions. Condition (2) ensures that B covers all source pairs involved in any outgoing transition, while condition (3) propagates this property to target states, ensuring that the image of any transition remains within the bisimulation witness.

Then we can conclude that this transducer is efficiently projectable if such a witness is found.

THEOREM 4.4 (CORRECTNESS OF PROJECTION). Let $T = (S, S_0, S_f, \Delta_S, \emptyset, \emptyset, \emptyset)$ be a synchronous NetKAT transducer. If there exists a bisimulation witness B , then T is efficiently projectable.

Relational Splitting. After defining the class of transducers that can be efficiently projected, the next step is to transform a general $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$ transducer into one that belongs to this class. The key challenge lies in discovering a bisimulation witness. However, the bisimulation witness defined in the previous section may not exist for all transducers T . Therefore, we aim to split the transducer by states or transitions until a bisimulation witness can be identified for every state s .

A natural approach is to compute the maximum bisimulation at each state and then split states according to the equivalence classes of this relation. These equivalence classes form a bisimulation witness for the resulting transducer, thereby ensuring that it is *efficiently projectable*. Moreover, this approach yields a transducer with the minimal number of states after splitting. We describe this method in the Appendix.

However, this approach is inefficient in practice, as computing the global bisimulation relation is computationally expensive. Prior work [Fisler and Vardi 2002] has shown that computing

the maximum bisimulation relation for a symbolic model can be more costly than verifying an invariant for that model. This overhead becomes prohibitive when handling industrial networks with thousands of devices.

To avoid this cost, we propose a "local" approach that computes equivalence classes on a per-transition basis. Specifically, we check whether (x_1, y) and (x_2, y) reach the same (x', y') in the next state. This yields the following equivalence relation:

$$x_1 \equiv_{s,y} x_2 \triangleq \forall s', ((x_1, y), (x', y')) \in \Delta(s, s') \iff ((x_2, y), (x', y')) \in \Delta(s, s')$$

It is straightforward to verify that this relation defines an x -bisimulation for transducer T . This localized x -bisimulation enables us to generate bisimulation witnesses per transition. Our experiments in Section 5 demonstrate that this approach yields a $2.5\times$ speedup over the global bisimulation method. We adopt this local splitting approach in the paper, and the resulting transducer construction based on this equivalence relation is defined as follows:

Step 1: Transition-Based Partitioning. The first step is to partition each transition relation $\Delta_S(s, s')$ into a set of subrelations r' such that each subrelation's input pairs is indistinguishable towards y -tape. i.e., for all $(x_1, y), (x_2, y) \in \text{dom}(r')$, x', y' we have

$$((x_1, y), (x', y')) \in r' \iff ((x_2, y), (x', y')) \in r'$$

We do not prescribe a concrete algorithm for this partitioning, as efficient strategies may depend heavily on the structure of the transition r . In principle, a naive partitioning can always be achieved by enumerating all x values corresponding to each fixed y , although this may lead to exponential blowup. In practice, more optimized approaches are often possible. For example, one such strategy is discussed in Section 5, where we exploit the low-level BDD representation used in our implementation to enable efficient symbolic partitioning.

Formally, we suppose that we have partition function $P : S \times S \rightarrow 2^{(Pk \times Pk) \times (Pk \times Pk)}$ such that:

- For each pair of states (s, s') , each $r \in P(s, s')$ satisfies a localized bisimulation condition required for efficient projection. More precisely, for every $r \in P(s, s')$, the following holds for all $(x_1, y), (x_2, y) \in \text{dom}(r)$, x', y' we have

$$((x_1, y), (x', y')) \in r \iff ((x_2, y), (x', y')) \in r$$

This condition ensures that all pairs (x, y) appearing as inputs to the transition relation r' are indistinguishable towards the output at next states.

- The partition covers the transition relation:

$$\left(\bigcup_{r \in P(s, s')} r \right) = \Delta(s, s').$$

Step 2: Bisimulation Witness Generation. The ultimate goal of splitting is to transform the transducer into one that is efficiently projectable, with bisimulation witnesses serving as the guarantee. In this step, we aim to assign each state $s \in S$ a set of bisimulation witnesses such that, in the final split transducer, the states take the form (s, b) , where the bisimulation witness of (s, b) is b . Formally, we define the target function $\mathcal{B} : S \rightarrow 2^{2^{Pk \times Pk}}$ as follows:

$$\mathcal{B}(s) = \begin{cases} \{\text{dom}(r) \mid \exists s' \in S, r \in P(s, s')\}, & \text{if } s \notin S_f \\ \{Pk \times Pk\}, & \text{if } s \in S_f \end{cases}$$

The idea behind this construction is that for every non-final state s , and every transition $r \in P(s, s')$, there exists a domain-covering (one that satisfy property (2)) witness $\text{dom}(r)$ at state

$(s, \text{dom}(r))$. For final states, we assume without loss of generality³ that there are no outgoing transitions. Thus, domain coverage is not a concern, and we may safely assign $\mathcal{B}(s)$ to be the singleton set $Pk \times Pk$ to ensure coverage for any incoming transition (i.e., range coverage).

Combined with the previous step of transition partitioning, this witness generation ensures that in the final split transducer, each witness b at state (s, b) satisfies both condition (1) (bisimulation witness) and condition (2) (domain coverage). In the next step, we will ensure condition (3) (range coverage) through output restriction.

Step 3: State Splitting and Output Restriction. Using $\mathcal{B}(s)$, we now construct a new transducer where states are paired with their bisimulation witness. Define the split transducer:

$$T^P \triangleq (S^P, S_0^P, S_f^P, \Delta_S^P, \emptyset, \emptyset, \emptyset)$$

as follows:

- $S^P \triangleq \{(s, b) \mid s \in S, b \in \mathcal{B}(s)\}$.
- $S_0^P \triangleq \{(s_0, b) \mid s_0 \in S_0, b \in \mathcal{B}(s_0)\}$.
- $S_f^P \triangleq \{(s_f, b) \mid s_f \in S_f, b \in \mathcal{B}(s_f)\}$.
- Transition relation:

$$\Delta_S^P((s_1, b_1), (s_2, b_2)) \triangleq \bigcup_{b_1 = \text{dom}(r) \wedge r \in P(s_1, s_2)} \{(x, y), (x', y') \in r \mid (x', y') \in b_2\}$$

The goal of this construction is that, attached to each state (s, b) after splitting, the packet relation b serves as the bisimulation witness for that state, therefore an efficient projection is then doable. As discussed above, without output restriction, the split transition would be:

$$\Delta_S^P((s_1, b_1), (s_2, b_2)) \triangleq \bigcup_{b_1 = \text{dom}(r) \wedge r \in P(s_1, s_2)} r$$

In this form, the b in each state (s, b) satisfies condition (1) and condition (2). Now, we apply the final output restriction step to ensure condition (3) (range coverage).

Although the output restriction prunes the output of each transition, it does not violate bisimulation conditions (1) and (2). For condition (2), the filtered transition is a subset of the unfiltered one, so domain coverage still holds. For condition (1), the key insight is this: because we adopt a x -bisimulation relation that they are behavioural indistinguishable in terms of output transition behavior, applying uniform truncation on their output transitions will preserve another uniform output transition behavior, thus x -bisimulation is retained. We thus conclude with the following theorem:

THEOREM 4.5 (CORRECTNESS OF SPLITTING). *Let $T = (S, S_0, S_f, \Delta_S, \emptyset, \emptyset, \emptyset)$ be a synchronous NetKAT transducer. Without loss of generality, we suppose that final states have no outgoing transitions. Then the split transducer T^P constructed as above satisfies:*

- $L(T^P) = L(T)$
- T^P is in the class of efficiently projectable transducers.

Then we summarize all of the compilation pipeline to the projected automaton $\mathcal{A}(K \triangleright R)$ as:

$$\mathcal{A}(K \triangleright R) \triangleq |((\mathcal{A}(R)|_{\mathcal{A}(K)})^S)^P|$$

Having constructed the projected automaton $\mathcal{A}(K \triangleright R)$ from the pipeline, we obtain an automaton of the form as a standard NetKAT automaton. This allows us to complete the verification of relational specifications $K_1 \triangleright R_1 = K_2 \triangleright R_2$. To this end, we apply two standard procedures:

³This holds trivially for $(\mathcal{A}(R)|_{\mathcal{A}(K)})^S$, as shown in the appendix.

determinization, followed by *bisimulation checking*. These procedures follow the same methodology as prior works [Moeller et al. 2024; Smolka et al. 2015], and are deferred to the Appendix.

5 Evaluation

In this section, we describe the implementation of our Relational NetKAT compiler and compare its performance against existing tools, such as Batfish and Rela, for verification tasks that these other tools support. Our goal is to quantify the overhead of the more general approach of RN. We also benchmark RN's performance on tasks that are uniquely supported by RN and quantify the value of our optimizations techniques. Our artifact is available at [Xu et al. 2025].

5.1 Implementation

Our implementation has approximately 3000 lines of OCaml code, includes the following key components:

- (1) A compiler that translates a NetKAT expression K and a Relational NetKAT expression R into a NetKAT automaton $\mathcal{A}(K \triangleright R)$.
- (2) An equivalence checker that checks the equivalence of two NetKAT automata.
- (3) An interface for translating input/output formats of Batfish and Rela into our representation.

$\mathcal{A}(K \triangleright R)$ Compiler. We follow the compilation pipeline outlined in Section 4. At the lowest level, we encode packets using binary representations and use Binary Decision Diagrams (BDDs) for symbolic manipulation. Specifically, we use the [MLBDD](#) library, where each boolean variable corresponds to the value of a particular field bit.

Since automaton transitions involve pairs such as (x, x') and $((x, y), (x', y'))$, we interleave the bit positions of the four packets within the BDD variable ordering. Concretely, for each bit index k , we assign positions $6k$, $6k + 1$, $6k + 2$, and $6k + 3$ to the bits of x , x' , y , and y' respectively, while reserving $6k + 4$ and $6k + 5$ for BDD composition. This encoding is especially efficient for common cases like identity relations or slight field modifications.

After BDD encoding, we implement the functions `delta_k`, `delta_r`, and `delta_kr` to construct the automata $\mathcal{A}(K)$, $\mathcal{A}(R)$, and $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$ respectively. We apply a "reachability pruning" optimization to prune unreachable states before performing relational splitting, reducing the number of (y, x) pairs that must be considered. With this reachability optimization, we are able to do an emptiness check using the function `emptiness_check` at this phase.

Next, we implement a naive but effective BDD-based splitting strategy to ensure that each y value is associated only with indistinguishable x values. To achieve this, we reorder the BDD variable layout from (x, x', y, y') — i.e., $6k, 6k + 1, 6k + 2, 6k + 3$ — to (y, x, y', x') — i.e., $6k + 1, 6k + 3, 6k, 6k + 2$. This reordering enables us to analyze, for each y , whether multiple x values appear before the next y variable is encountered in the BDD. If so, we split the BDD branches; otherwise, we retain the structure. Recursively applying this process yields a partition satisfying the criteria for splitting.

We then combine this splitting logic with the algorithms described in Section 4, implementing the procedures `generate_all_transition` and `simplify_all_transition` for relational splitting and projection. Finally, we support automaton determinization through the function `determinization`, completing the compilation pipeline.

Equivalence Checker. The equivalence checker and counter-example generation follows the bisimulation-based approach introduced by Moeller et al. [2024]. Specifically, we implement the bisimulation procedure via the function `bisim`, which symbolically compares two deterministic automata for language equivalence. To generate counterexamples when equivalence fails, we compile the symmetric difference of two NetKAT expressions $e_1 \oplus e_2$ (in our case $\mathcal{A}(K_1 \triangleright R_1) \oplus$

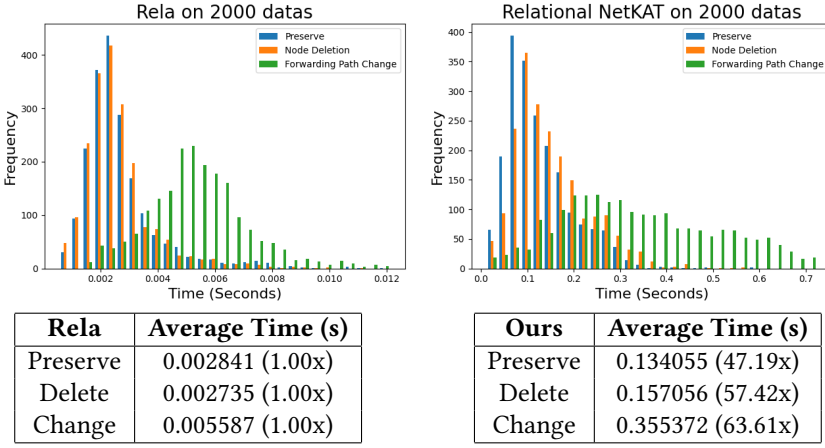


Fig. 5. Arithmetic runtime comparison between Relat and our system across three verification goals

$\mathcal{A}(K_2 \triangleright R_2)$), implemented by the function `symmetric_difference`. This process enables us to identify traces accepted by one expression but not the other.

Interfaces for Batfish and Relat. To interface with Batfish and Relat, the first step is to encode packet structures into BDDs. We adopt two different encoding strategies for packet fields. For source/destination IP addresses and ports, we use a bit-aligned binary encoding that matches their natural layout in packets. This representation supports efficient prefix matching (e.g., for IP prefixes such as $10.20.1.0/24$) and is well-suited to forwarding table lookups. For other fields such as `loc = A` or `typ = ssh`, we assign unique identifiers to each field value and encode them into binary form. This hybrid approach avoids wasting bits on unused values while preserving efficient predicate evaluation. Combined, these strategies enable us to implement backends compatible with both Batfish and Relat.

For Batfish, RN consumes its output as input. Batfish ingests network configuration and exposes the network’s routing and topology information using higher-level APIs. We use the `routes()` API to extract routing tables for each node, and `layer3Edges()` API to extract the network’s topology. Interface-level properties and access control lists are retrieved using `interfaceProperties()` and `namedStructures()` APIs. IP tunnel information is extracted using `ipsecEdges()` API. Since Batfish currently lacks APIs that yield firewall and NAT, we directly parse configuration files to extract these features as needed.

For Relat, Relational NetKAT acts as a natural extension of its IR. In particular, Relat-style queries of the form $P_1 \triangleright R_1 = P_2 \triangleright R_2$ are translated into Relational NetKAT expressions. We compile path sets P_1 and P_2 —representing networks annotated with IP constraints—into pre- and post-network NetKAT expressions K . Similarly, we promote path relations R_1 and R_2 to relational programs.

5.2 Comparison to Relat

Relat [Xu et al. 2024] is a relational network verification tool for verifying path changes in networks that do not transform packets. It models the network as a set of DAGs annotated with source and destination IP constraints. As shown earlier, our language subsumes the syntax of Relat’s intermediate representation, enabling a direct comparison.

Xu et al. [2024] released a dataset with pre- and post-network paths from Alibaba’s global backbone. Because the data is not accompanied by specific verification goals, we consider three common verification goals based on the list of changes in the paper’s appendix:

Table 1. Runtime of Batfish and RN on Example Batfish Benchmarks

Forwarding Change Validation		
Function	Batfish (s)	Ours (s)
traceroute 1	1.013	0.641
reachability 1	0.440	1.391
differential 1	0.396	1.218
reachability 2	0.254	0.875
differential 2	0.246	0.391
traceroute 2	0.429	0.140
traceroute 3	0.372	0.344
reachability 3	0.377	1.046
differential 3	0.378	1.125
reachability 4	0.234	0.782
differential 4	0.398	0.625
Average	0.414	0.782(1.8x)

Hybrid Cloud Network		
Function	Batfish (s)	Ours (s)
traceroute 1	0.399	40.657
traceroute 2	0.230	3.406
traceroute 3	0.222	2.797
traceroute 4	0.380	2.281
traceroute 5	0.212	2.828
reachability 1	0.224	7.656
Average	0.278	9.271(33.3x)

- **Preserve:** Verify equivalence between the original and updated DAGs.
- **Node Deletion:** Randomly delete one node and its associated traffic, then verify equivalence.
- **Forwarding Path Change:** Redirect traffic through a randomly selected node to another random node (e.g., changing the path $A \rightarrow B \rightarrow C$ to $A \rightarrow D \rightarrow C$), and verify equivalence.

The dataset includes 2460 devices and 21,112 traffic DAGs. We randomly selected 2000 DAGs and applied each verification tasks above. As shown in Figure 5, our system is approximately 50–60 times slower than Relat. This slowdown stems from two factors:

- We use BDDs to encode location and IP headers, resulting in heavier symbolic manipulation. In contrast, Relat uses a single-character symbol per location.
- Our verification relies on symbolic automata due to the large packet space. Symbolic transitions are not necessarily disjoint, requiring $O(n)$ checks for disjointness, whereas regular automata alphabets are disjoint and allow $O(\log n)$ disjointness checks.

Despite the slowdown, our system exhibits acceptable performance (sub-second validation time). We deem that RN’s slower relative performance is worthwhile given its support for richer trace relations and stronger guarantees for change types that both tools support. Unlike Relat, which considers concrete packets, RN verifies changes over extremely large packet spaces ($> 2^{32}$ packets).

5.3 Comparison to Batfish

Batfish is a widely used network verification tool that supports a broad range of single-snapshot verification tasks, along with limited forms of relational verification. To compare RN with Batfish, we replicate forwarding analyses from two tutorial examples: *Forwarding Change Validation* and *Hybrid Cloud Network*. These examples demonstrate Batfish’s capabilities in relational verification and its scalability on realistic network topologies. The first example features a network with 9 devices and approximately 8.5k lines of JSON-based routing configuration. The second example involves a more complex network with 17 devices and over 200k lines of routing policies.

Our system successfully reproduces a wide range of Batfish-supported queries. In particular, we focus on data plane analysis queries which are:

- (1) *reachability* – Determines whether packets satisfying header constraints H (e.g., $\text{dst.ip} = 10.0.0.1$) can traverse a path matching path constraints P (e.g., from A to C via B).

Table 2. Relational Verification Queries Only Supported by Relational NetKAT

Hybrid Cloud Network		
Verification	Ours (s)	Specification
NAT unchanged 1	26.152	$K_{pre} \triangleright Id(alltraces(src.ip \neq NAT_ip \cdot dst.ip \neq NAT_ip))$ $= K_{post} \triangleright Id(alltraces(src.ip \neq NAT_ip \cdot dst.ip \neq NAT_ip))$
NAT unchanged 2	21.141	$K_{pre} \triangleright Id((\neg(NAT_entry) \circ dup)^*)$ $= K_{post} \triangleright Id((\neg(NAT_entry) \circ dup)^*)$
NAT changed 1	7.469	$K_{post} \triangleright Id(\overline{loc = outer \cdot dst.ip = public_ip} \circ alltraces \circ \overline{loc = inner}) = \emptyset$
Tunneling 1	28.687	$K_{pre} = K_{post}$
Tunneling 2	20.515	$K_{pre} \triangleright Map(encrypt_it + cleartext, alltraces) = K_{post}$
Average	20.792	

- (2) *traceroute* – Similar to *reachability*, but without user-defined path constraints; only the start location and header constraint are specified.
- (3) *differentialReachability* – Verifies whether packets satisfying H reach a path P in one snapshot but are filtered in another.

We specify general reachability in relational NetKAT as:

$$reachability = Filter(\bar{1})Delete(alltrace)Insert(havoc)Filter(\bar{1})$$

This relation maps all traces in the network into its input/output pair, where the verification task is: is formulated as:

$$K_{pre} \triangleright reachability = K_{post} \triangleright reachability$$

The *traceroute* query focuses the *reachability* query with a header constraint H and a path constraint P . We can express these constraints using a packet predicate P_h and NetKAT expression K_p , respectively. For instance, in the [Forwarding Change Validation](#) example, one goal is to ensure no flow traverses "core1" after a configuration change. This corresponds to $K_p = alltrace \circ loc \leftarrow core1 \circ dup \circ alltrace$. A constraint like $dst.ip = "54.191.42.182"$ from the [Hybrid Cloud Network](#) is similarly encoded as a packet predicate. We define a strengthened reachability as:

$$reachability' = Filter(P_h)Delete(K_p)Insert(havoc)Filter(\bar{1})$$

This can be used for input-output reachability queries, but to extract full trace sets (as in Batfish's *traceroute*), we define:

$$traceroute = Filter(P_h)Map(\bar{1}, K_p)$$

Finally, differential reachability across network snapshots is formulated as:

$$K_{pre} \triangleright reachability' = K_{post} \triangleright reachability'$$

This strengthens Batfish's version by enforcing output and end-location equivalence.

Table 1 shows the performance of Batfish and RN on the forwarding analyses in the two examples. While RN shows comparable performance to Batfish for most analyses, it has much higher runtime for some, where BDD and initialization overhead dominate. For instance, in the hybrid cloud benchmark, Traceroute 1 takes over 40s in our system due to symbolic automata and BDD initialization for over 200k lines of routing policies, whereas Batfish completes parsing in under 1s. We cache automata computations using hash tables to reduce recomputation costs.

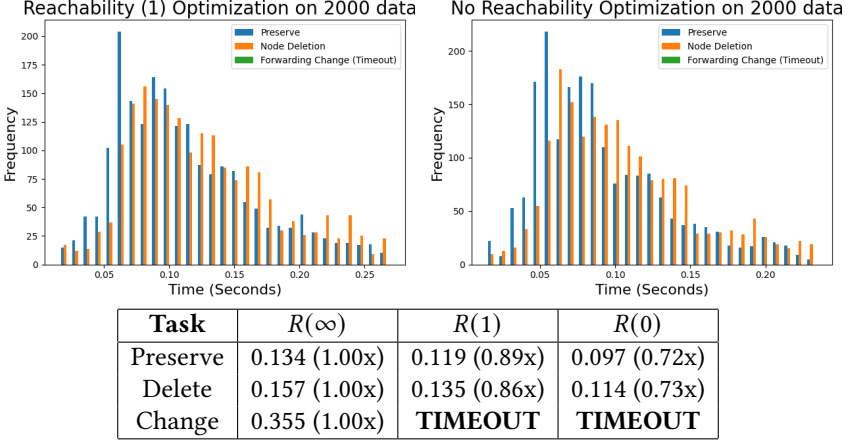


Fig. 6. Comparison of verification times under different levels of reachability pruning. TIMEOUT > 1000s.

5.4 Relational Verification for Networks with Transformations

Our system supports relational verification tasks that no current tool (including Batfish and Rela) can express. Table 2 lists several such relational queries and our system’s performance for them on the same network as the Hybrid Cloud example above. The queries use NAT (Network Address Translation) and tunneling.

NAT queries. NAT is often deployed to bridge internal (private) and external (public) networks. It rewrites packet headers such that internal hosts appear under public IPs. In our experiment, the network operator disables the NAT entry to isolate the internal network from external traffic.

In networks with NAT, it is hard to get the exact change as changes of header also may change the forwarding path, leading to another path which is only able to be described with certain knowledge on the forwarding tables. We handle NAT-related queries by separately verifying the unchanged and affected portions of the network. In NAT unchanged 1, we confirm that traces not containing NAT IPs remain unaffected, and in NAT unchanged 2, we confirm that traces not going through NAT entries remain unaffected. While NAT changed 1 checks that without NAT, traces coming from outside cannot reach a private destination with a public NAT ip.

Tunneling queries. The tunneling queries analyze changes from a flat, untunneled network to a tunneled configuration, as discussed in Section 3. Our system encodes symbolic equivalence between encrypted and decrypted behaviors. Tunneling 1 verifies equivalence when all IP tunnels connect adjacent nodes, which is the original example’s IPsec tunneling setting. In contrast, Tunneling 2 encrypts traffic at randomly selectively nodes, and verifies the correspondence of the decrypted and encrypted network.

5.5 Impact of Optimizations

Our verification engine has two primary optimizations: reachability pruning and a custom splitting algorithm. We now evaluate the performance impact of these optimizations.

Reachability Pruning. This optimization is applied after composing the $(\mathcal{A}(R)|_{\mathcal{A}(K)})^s$ and before using the splitting algorithm. It prunes unreachable configurations thereby improving the efficiency of splitting. We denote different levels of reachability pruning using $R(\infty)$, $R(1)$, and $R(0)$. $R(\infty)$ computes all reachable pairs exhaustively; $R(0)$ applies no pruning at all, assuming

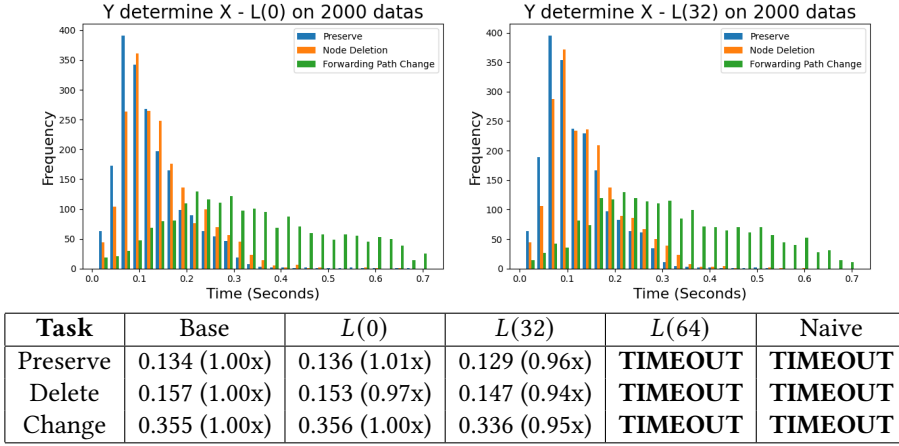


Fig. 7. Comparison of verification times under different splitting strategies. TIMEOUT > 1000s.

that all configurations are reachable, and $R(1)$ is the overapproximation obtained from $R(0)$ doing one step of reachability pruning.

As shown in Figure 6, disabling reachability pruning $R(0)$ yields faster performance in the PRESERVE and DELETE tasks due to reduced preprocessing overhead. In these cases, the packet relation is typically simple (e.g., $\bar{1}$ or $\overline{loc = B}$) and can be projected efficiently.

However, for CHANGE tasks that involve nontrivial path changes (e.g., from $A \rightarrow C \rightarrow B$ to $A \rightarrow D \rightarrow B$), pruning becomes essential. Without pruning, $R(0)$ loses track of the (x, y) relation after leaving A , and $R(1)$ loses it after passing through C or D on the way to B . Consequently, even if transitions are identity mappings (i.e., $x = x' \wedge y = y'$), the splitting procedure on $((x, y), (x', y'))$ must consider an exponential number of x values due to lack of constraint on (x, y) , leading to timeouts (> 1000s).

Splitting Algorithm. A key technical contribution of this paper is the novel splitting algorithm we propose for efficient automata projection. We compare its performance against several alternatives. Our design uses the observation that if the y -tape (i.e., output projection) can uniquely determine a group of x -tape values that are indistinguishable with respect to the next output transition, then we can significantly reduce the number of splits required. However, it is also possible to generate a correct projected automaton using either (1) a naive splitting strategy that enumerates all possible x values or (2) a more aggressive strategy where y uniquely determines x .

To evaluate these alternatives, we test each method on automata augmented with unused “hidden” field lengths of 0, 32, and 64 bits. These fields do not affect our current implementation (serving as a baseline, labeled **Base**), but they do impact the strategies that require y to determine x , which we denote as $L(0)$, $L(32)$, and $L(64)$. The naive strategy simply times out (i.e., exceeds 1000 seconds) on all test cases and is labeled **Naive**, as shown in Figure 7.

Our implementation employs hash-consing to memoize BDD splits, avoiding redundant computations when the same BDD structure appears. Even so, this overhead is only small when the hidden length is small (e.g., 0 or 32). However, as the hidden field length increases (e.g., to 64 bits), the splitting overhead becomes exponential, resulting in timeouts (>1000s).

This experiment confirms that our splitting algorithm is both efficient and necessary for practical automata construction in the presence of large packet spaces.

Maximum vs. Localized Bisimulation. As discussed in Section 4, we adopt a *localized* approach that computes the equivalence classes of (x, y) only up to the next transition. This approach is

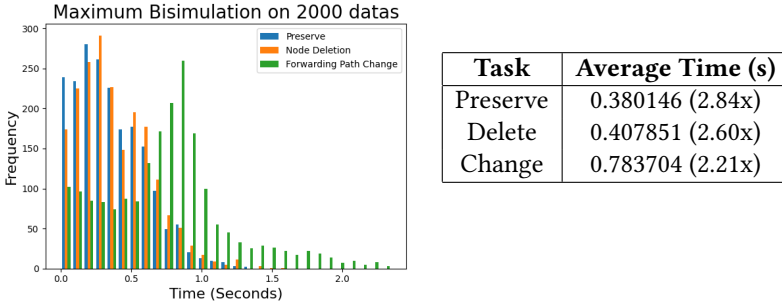


Fig. 8. Runtime of Maximum bisimulation.

significantly more efficient than the *maximum bisimulation* method because it avoids computing the full global bisimilarity relation and allows us to proceed with automata splitting immediately. The complete algorithm for the global approach is provided in the Appendix. Figure 8 presents the experimental results, showing an overall speedup of more than $2.2\times$ across all cases.

5.6 Other Optimizations

We experimented with and implemented several engineering optimizations that yielded significant performance improvements in compiling and manipulating NetKAT transducers. Due to a limit of space and the fact that these optimizations are not at the core of our theory, we put the experiments and verifications for these optimization results in our artifact.

Hash Table Memoization. We memoized results in key procedures such as derivatives, cross-product construction, and BDD splitting, which are often invoked repeatedly with identical inputs. This optimization led to a more than $100\times$ speedup on the Rela benchmark.

Efficient Routing Table Encoding. While different routing encodings ultimately yield the same BDDs, compilation time varies significantly. We replaced the naive sequential insertion of routing entries with a binary tree organization sorted by destination IPs. This change reduced compile time in the Hybrid Cloud Network case from 30+ minutes to under 60 seconds.

Implicit vs. Explicit Location Encoding. We opted to encode location information *within* the BDDs rather than using explicit NetKAT states. This implicit representation yields a more than $4\text{--}5\times$ speedup over the explicit alternative on the Rela benchmark.

Set-Based Union Operator. Rather than encoding expressions like $e_1 + e_2 + \dots + e_n$ as nested binary unions, we implemented a set-based construction $\{e_1, e_2, \dots, e_n\}$. This ensures that semantically equivalent expressions are also syntactically equivalent, reducing automata state explosion from factorial to linear size in some cases.

Direct Automata Compilation from JSON.. We explored bypassing NetKAT parsing altogether by compiling automata directly from Rela’s JSON output. However, due to the need for explicit location encoding (since Antimirov derivatives don’t apply to the JSON format), this approach incurred slowdown compared to our current method.

Future Optimizations. The following optimizations could lead to further performance gains but are beyond the current engineering scope of our work:

- **MLBDD Library Enhancements.** Although MLBDD is the best OCaml BDD library we found, it still lacks several key features: 1) Dynamic variable reordering, 2) Garbage collection,

3) Parallelization. These improvements could significantly enhance compilation scalability and runtime.

- **Reimplementation in C/C++.** OCaml, while productive and concise, lacks high-performance transducer libraries and runtime features available in C/C++. Rewriting our core compiler in C/C++ could unlock further performance improvements, especially for low-level data structures and BDD manipulations.

6 Related Work

As mentioned earlier, there has been much past work on single-snapshot network verification [Backes et al. 2019; Beckett et al. 2017; Fogel et al. 2015; Jayaraman et al. 2019; Kakarla et al. 2020; Kazemian et al. 2012; Khurshid et al. 2013; Mai et al. 2011; Zeng et al. 2014] but little work aside from Rela [Xu et al. 2024] (discussed in the introduction) on multi-snapshot change validation.

Algorithms for Automata Construction. There are two main approaches to constructing NetKAT automata. One line of work [Smolka et al. 2015] adopts Antimirov partial derivatives [Antimirov 1996]; the other line of work [Foster et al. 2015; Moeller et al. 2024] is based on Brzozowski derivatives [Brzozowski 1964]. The essential distinction is that Antimirov derivatives yield nondeterministic automata (NFA), while Brzozowski derivatives construct deterministic automata (DFA) directly.

We adopt Antimirov derivatives for three key reasons: 1) The number of Antimirov derivatives is bounded by the syntactic size of the expression, yielding compact NFAs. 2) Even if our $\mathcal{A}(K)$ and $\mathcal{A}(R)$ constructions appear deterministic, their cross-product automaton after synchronization is not guaranteed to remain deterministic. And 3) prior work [Pous 2015] finds Antimirov-based symbolic algorithms to be more efficient than Brzozowski-based alternatives for Kleene Algebra with Tests (KAT). Thus, we construct automata using Antimirov derivatives and then apply determinization in a separate step.

There are also several choices when it comes to deciding equivalence between symbolic automata and/or KAT terms [Bonchi and Pous 2013; D’Antoni and Veanes 2014, 2017; Pous 2015]. Our implementation adopts the latest bisimulation-based symbolic approach used in the KATch system [Moeller et al. 2024]. However, our implementation deviates from KATch in other ways. While KATch uses Forwarding Decision Diagrams (FDDs) to encode transition relations [Moeller et al. 2024; Smolka et al. 2015], we used Binary Decision Diagrams (BDDs) for two reasons. First, while FDDs are optimized for binary transition relations over packet pairs (x, x') , Relational NetKAT uses two-tape transitions of the form $((x, y), (x', y'))$, which we were not able to represent efficiently with FDDs, especially when reordering is needed during splitting. Second, FDDs are designed to handle atomic tests such as $f = v$, and updates like $f \leftarrow v$ efficiently, but have no compact representation for richer constructs like *havoc*, which show up often in our change specifications.

7 Conclusion

Relational NetKAT (RN) is a compositional language for specifying a wide range of network changes, including updates to forwarding paths and modifications to packet-processing behavior such as tunnels, NATs, and firewall rules. These changes are expressed as trace relations, mapping traces in the pre-change network to those in the post-change network. To verify compliance with such specifications, RN compiles trace relations into NetKAT transducers and applies them to NetKAT automata of both pre- and post-network. To avoid potential blowup in packet space, we design a relational splitting technique that exploits the observation that real-world changes are usually “small.” Our OCaml implementation [Xu et al. 2025] and case studies show that RN captures a wide range of realistic network updates and enables efficient verification at industrial scale.

Data-Availability Statement

The companion artifact [Xu et al. 2025] includes the ocaml code.

Acknowledgments

The authors wish to thank the members of Princeton PL group and the anonymous POPL reviewers for their helpful feedback. We are also grateful for the support provided by Bloomberg through the Bloomberg Infrastructure & Security Fellowship, which has supported Han Xu’s doctoral studies. This work is also supported in part by NSF awards 2219862, 2219863 and 2312539.

References

- Carolyn Jane Anderson, Nate Foster, Arjun Guha, Jean-Baptiste Jeannin, Dexter Kozen, Cole Schlesinger, and David Walker. 2014a. NetKAT: semantic foundations for networks. *SIGPLAN Not.* 49, 1 (Jan. 2014), 113–126. doi:10.1145/2578855.2535862
- Carolyn Jane Anderson, Nate Foster, Arjun Guha, Jean-Baptiste Jeannin, Dexter Kozen, Cole Schlesinger, and David Walker. 2014b. NetKAT: Semantic Foundations for Networks. In *ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 113–126.
- Valentin Antimirov. 1996. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science* 155, 2 (1996), 291–319. doi:10.1016/0304-3975(95)00182-4
- John Backes, Sam Bayless, Byron Cook, Catherine Dodge, Andrew Gacek, Alan Hu, Temesghen Kahsai, Bill Kocik, Evgenii Kotelnikov, Jure Kukovec, Sean McLaughlin, Jason Reed, Neha Rungta, John Sizemore, Mark Stalzer, Preethi Srinivasan, Pavle Subotic, Carsten Varming, and Blake Whaley. 2019. *Reachability Analysis for AWS-Based Networks*. 231–241. doi:10.1007/978-3-030-25543-5_14
- Ryan Beckett, Aarti Gupta, Ratul Mahajan, and David Walker. 2017. A General Approach to Network Configuration Verification. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication* (Los Angeles, CA, USA) (*SIGCOMM ’17*). Association for Computing Machinery, New York, NY, USA, 155–168. doi:10.1145/3098822.3098834
- Filippo Bonchi and Damien Pous. 2013. Checking NFA equivalence with bisimulations up to congruence. *SIGPLAN Not.* 48, 1 (Jan. 2013), 457–468. doi:10.1145/2480359.2429124
- Janusz A. Brzozowski. 1964. Derivatives of Regular Expressions. *J. ACM* 11, 4 (Oct. 1964), 481–494. doi:10.1145/321239.321249
- Loris D’Antoni and Margus Veanes. 2014. Minimization of symbolic automata. *SIGPLAN Not.* 49, 1 (Jan. 2014), 541–553. doi:10.1145/2578855.2535849
- Loris D’Antoni and Margus Veanes. 2017. Forward Bisimulations for Nondeterministic Symbolic Finite Automata. In *Tools and Algorithms for the Construction and Analysis of Systems*, Axel Legay and Tiziana Margaria (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 518–534.
- Kathi Fisler and Moshe Y. Vardi. 2002. Bisimulation Minimization and Symbolic Model Checking. *Form. Methods Syst. Des.* 21, 1 (July 2002), 39–78. doi:10.1023/A:1016091902809
- Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. 2015. A General Approach to Network Configuration Analysis. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. USENIX Association, Oakland, CA, 469–483. https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/fogel
- Nate Foster, Dexter Kozen, Mae Milano, Alexandra Silva, and Laure Thompson. 2015. A Coalgebraic Decision Procedure for NetKAT. *SIGPLAN Not.* 50, 1 (Jan. 2015), 343–355. doi:10.1145/2775051.2677011
- Karthick Jayaraman, Nikolaj Bjørner, Jitu Padhye, Amar Agrawal, Ashish Bhargava, Paul-Andre C Bissonnette, Shane Foster, Andrew Helwer, Mark Kasten, Ivan Lee, Anup Namdhari, Haseeb Niaz, Aniruddha Parkhi, Hanukumar Pinnamraju, Adrian Power, Neha Milind Raje, and Parag Sharma. 2019. Validating datacenters at scale. In *Proceedings of the ACM Special Interest Group on Data Communication* (Beijing, China) (*SIGCOMM ’19*). Association for Computing Machinery, New York, NY, USA, 200–213. doi:10.1145/3341302.3342094
- Siva Kesava Reddy Kakarla, Alan Tang, Ryan Beckett, Karthick Jayaraman, Todd Millstein, Yuval Tamir, and George Varghese. 2020. Finding Network Misconfigurations by Automatic Template Inference. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, Santa Clara, CA, 999–1013. https://www.usenix.org/conference/nsdi20/presentation/kakarla
- Peyman Kazemian, George Varghese, and Nick McKeown. 2012. Header Space Analysis: Static Checking for Networks. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. USENIX Association, San Jose, CA, 113–126. https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/kazemian
- Ahmed Khurshid, Xuan Zou, Wenxuan Zhou, Matthew Caesar, and P. Brighten Godfrey. 2013. VeriFlow: Verifying Network-Wide Invariants in Real Time. In *NSDI*.

- Haohui Mai, Ahmed Khurshid, Rachit Agarwal, Matthew Caesar, P. Brighten Godfrey, and Samuel Talmadge King. 2011. Debugging the Data Plane with Anteater. In *Proceedings of the ACM SIGCOMM 2011 Conference* (Toronto, Ontario, Canada) (SIGCOMM '11). Association for Computing Machinery, New York, NY, USA, 290–301. doi:10.1145/2018436.2018470
- Mark Moeller, Jules Jacobs, Olivier Savary Belanger, David Darais, Cole Schlesinger, Steffen Smolka, Nate Foster, and Alexandra Silva. 2024. KATch: A Fast Symbolic Verifier for NetKAT. *Proc. ACM Program. Lang.* 8, PLDI, Article 224 (June 2024), 24 pages. doi:10.1145/3656454
- Damien Pous. 2015. Symbolic Algorithms for Language Equivalence and Kleene Algebra with Tests. *SIGPLAN Not.* 50, 1 (Jan. 2015), 357–368. doi:10.1145/2775051.2677007
- Jacques Sakarovitch. 2009. *Elements of Automata Theory*. Cambridge University Press.
- Steffen Smolka, Spiridon Eliopoulos, Nate Foster, and Arjun Guha. 2015. A fast compiler for NetKAT. *SIGPLAN Not.* 50, 9 (Aug. 2015), 328–341. doi:10.1145/2858949.2784761
- Han Xu, David Walker, Ratul Mahajan, and Zachary Kincaid. 2025. *Network Change Validation with Relational NetKAT (Artifact)*. doi:10.5281/zenodo.17650920
- Xieyang Xu, Yifei Yuan, Zachary Kincaid, Arvind Krishnamurthy, Ratul Mahajan, David Walker, and Ennan Zhai. 2024. Relational Network Verification. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (ACM SIGCOMM '24). Association for Computing Machinery, New York, NY, USA, 213–227. doi:10.1145/3651890.3672238
- Hongyi Zeng, Shidong Zhang, Fei Ye, Vimalkumar Jeyakumar, Mickey Ju, Junda Liu, Nick McKeown, and Amin Vahdat. 2014. Libra: divide and conquer to verify forwarding tables in huge networks. In *Proceedings of the 11th USENIX Conference on Networked Systems Design and Implementation* (Seattle, WA) (NSDI'14). USENIX Association, USA, 87–99.

Received 2025-07-10; accepted 2025-11-06